

软件架构模式

- 介绍
- 分层架构
- 模式描述
- 关键概念
- 模式示例
- 注意事项
- 模式分析
- 事件驱动架构
- 中介者拓扑
- 代理拓扑
- 注意事项
- 模式分析
- 微内核架构
- 模式描述
- 模式示例
- 注意事项
- 模式分析
- 微服务架构模式
- 模式描述
- 模式拓扑
- 避免依赖和编排
- 注意事项
- 模式分析
- 天基架构
- 模式描述
- 模式动力学
 - 消息传递网格
 - 数据网格
 - 处理网格
 - 部署经理
- 注意事项
- 模式分析
- 模式分析总结

介绍

对于开发人员来说，在没有正式架构的情况下开始编写应用程序的情况太常见了。如果没有清晰且定义良好的架构，大多数开发人员和架构师将诉诸事实上的标准传统分层架构模式（也称为 n 层架构），通过将源代码模块分离到包中来创建隐式层。不幸的是，这种做法通常会导致一堆无组织的源代码模块的集合，这些模块缺乏明确的角色、职责和相互关系。这通常被称为大泥球架构反模式。

缺乏正式架构的应用程序通常紧密耦合、脆弱、难以更改，并且没有明确的愿景或方向。因此，如果不充分了解系统中每个组件和模块的内部工作原理，就很难确定应用程序的架构特征。有关部署和维护的基本问题很难回答：架构是否可扩展？该应用程序有哪些性能特征？应用程序响应变化的容易程度如何？应用程序的部署特点是什么？该架构的响应能力如何？

架构模式有助于定义应用程序的基本特征和行为。例如，某些架构模式自然适合高度可扩展的应用程序，而其他架构模式自然适合高度敏捷的应用程序。为了选择满足您的特定业务需求和目标的架构模式，有必要了解每种架构模式的特征、优点和缺点。

作为一名架构师，您必须始终证明您的架构决策的合理性，特别是在选择特定架构模式或方法时。本报告的目的是为您提供足够的信息来做出决定并证明其合理性。

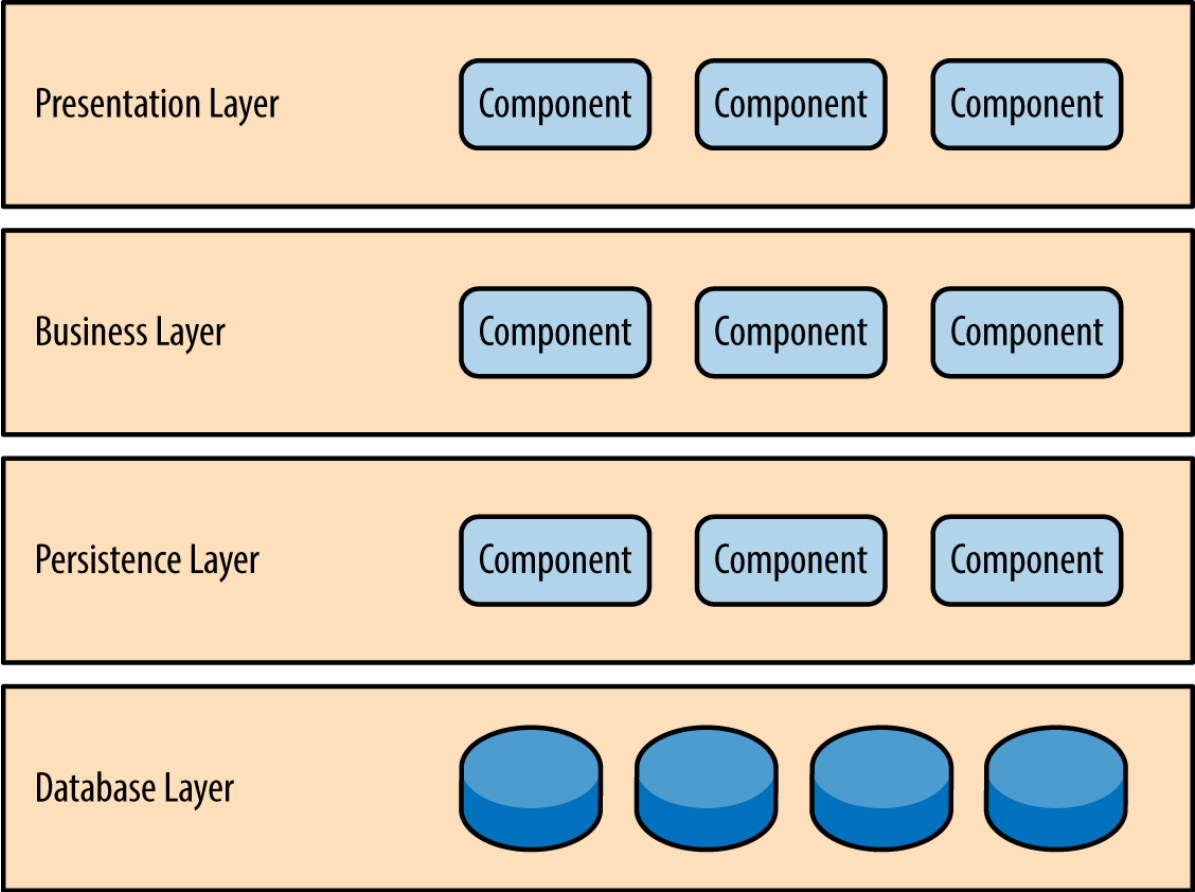
分层架构

最常见的架构模式是分层架构模式，也称为 n 层架构模式。该模式是大多数 Java EE 应用程序事实上的标准，因此被大多数架构师、设计人员和开发人员广泛熟知。分层架构模式与大多数公司中的传统 IT 通信和组织结构紧密匹配，使其成为大多数业务应用程序开发工作的自然选择。

模式描述

分层架构模式中的组件被组织成水平层，每一层在应用程序中执行特定的角色（例如，表示逻辑或业务逻辑）。尽管分层架构模式没有指定模式中必须存在的层的数量和类型，但大多数分层架构由四个标准层组成：表示层、业务层、持久层和数据库层（图1-1）。在某些情况下，业务层和持久层被组合成单个业务层，特别是当持久逻辑（例如，SQL或HSQL）嵌入到业务层组件中时。因此，较小的应用程序可能只有三层，而较大且更复杂的业务应用程序可能包含五层或更多层。

分层架构模式的每一层在应用程序中都有特定的角色和职责。例如，表示层将负责处理所有用户界面和浏览器通信逻辑，而业务层将负责执行与请求关联的特定业务规则。架构中的每一层都形成了围绕满足特定业务请求所需完成的工作的抽象。例如，表示层不需要知道或担心如何获取客户数据；它只需要以特定格式在屏幕上显示该信息。同样，业务层不需要关心如何格式化客户数据以在屏幕上显示，甚至客户数据来自哪里；它只需要从持久层获取数据，对数据执行业务逻辑（例如，计算值或聚合数据），然后将该信息传递到表示层。



1-1. 分层架构模式

分层架构模式的强大功能之一是组件之间的关注点分离。特定层内的组件仅处理属于该层的逻辑。例如，表示层中的组件仅处理表示逻辑，而驻留在业务层中的组件仅处理业务逻辑。这种类型的组件分类可以轻松地在架构中构建有效的角色和责任模型，并且由于定义良好的组件接口和有限的组件范围，还可以轻松地使用此架构模式开发、测试、管理和维护应用程序。

关键概念

请注意，在图 1-2 中，架构中的每一层都被标记为 关闭。这是分层架构模式中非常重要的概念。封闭层意味着当请求从一层移动到另一层时，它必须穿过其正下方的层才能到达该层下方的下一层。例如，来自表示层的请求必须首先经过业务层，然后到达持久层，最后到达数据库层。

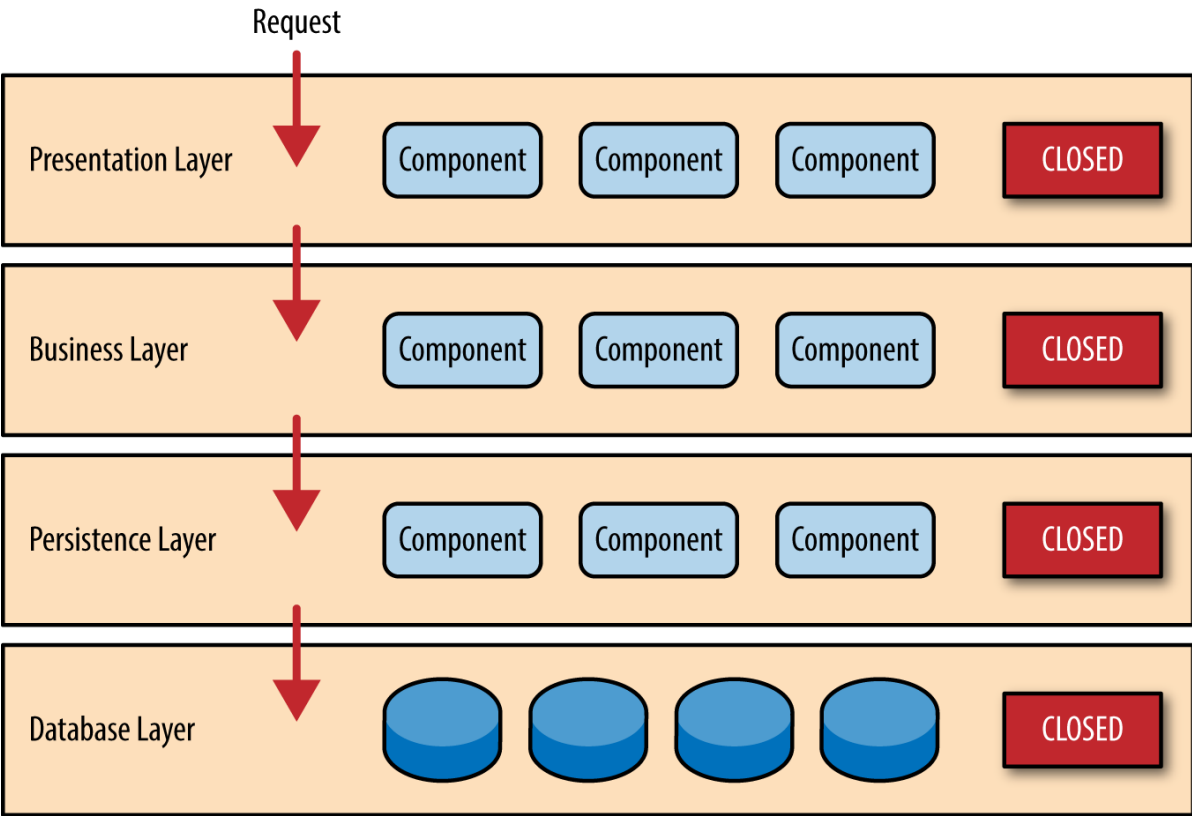


图 1-2.封闭层和请求访问

那么为什么不允许表示层直接访问持久层或数据库层呢？毕竟，从表示层直接访问数据库比仅仅通过一堆不必要的层来检索或保存数据库信息要快得多。这个问题的答案在于一个称为 隔离层的关键概念。

隔离概念的层意味着在架构的某一层中所做的更改通常不会影响或影响其他层中的组件：更改被隔离到该层内的组件，并且可能还隔离到另一个关联层（例如包含SQL）。如果允许表示层直接访问持久层，那么对持久层中的 SQL 进行的更改将影响业务层和表示层，从而产生一个非常紧密耦合的应用程序，组件之间存在大量相互依赖关系。改变这种类型的架构变得非常困难且昂贵。

隔离概念的层还意味着每一层都独立于其他层，因此很少或根本不了解架构中其他层的内部工作原理。要了解此概念的威力和重要性，请考虑进行大量重构工作，将表示框架从 JSP (Java Server Pages) 转换为 JSF (Java Server Faces)。假设表示层和业务层之间使用的契约（例如，模型）保持相同，则业务层不受重构的影响并且保持完全独立于表示层使用的用户界面框架的类型。

虽然封闭层有利于隔离层，因此有助于隔离架构内的更改，但有时开放某些层是有意义的。例如，假设您想要将共享服务层添加到包含由业务层内的组件访问的公共服务组件的体系结构（例如，数据和字符串实用程序类或审核和日志记录类）。在这种情况下，创建服务层通常是一个好主意，因为从架构上来说，它将对共享服务的访问限制在业务层（而不是表示层）。如果没有单独的层，就没有任何架构可以限制表示层访问这些公共服务，从而难以管理这种访问限制。

在此示例中，新的服务层可能位于 业务层下方，以指示无法从表示层访问此服务层中的组件。然而，这带来了一个问题，业务层现在需要经过服务层才能到达持久层，这根本没有意义。这是分层架构的一个古老问题，可以通过在架构中创建开放层来解决。

如图1-3所示，本例中的服务层被标记为开放，这意味着允许请求绕过该开放层并直接到达其下面的层。在下面的示例中，由于服务层是开放的，因此业务层现在可以绕过它并直接进入持久层，这是非常有意义的。

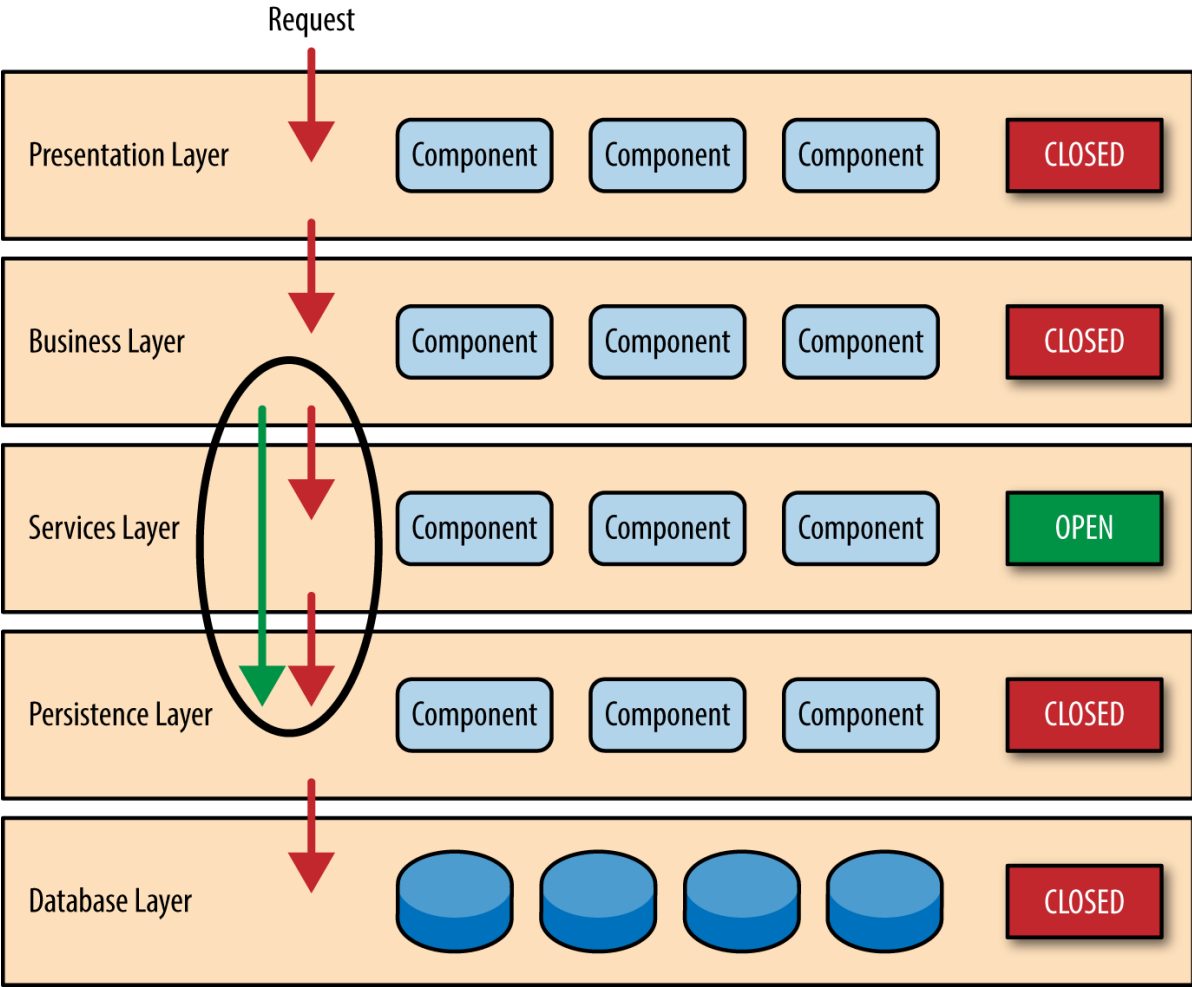


图 1-3.开放层和请求流程

利用开放层和封闭层的概念有助于定义架构层和请求流之间的关系，并为设计人员和开发人员提供必要的信息，以了解架构内的各个层访问限制。如果无法记录或正确传达架构中的哪些层是开放的和封闭的（以及原因），通常会导致紧密耦合且脆弱的架构，而这些架构非常难以测试、维护和部署。

模式示例

为了说明分层架构的工作原理，请考虑业务用户请求检索特定个人的客户信息，如图1-4所示。黑色箭头显示向下流动到数据库以检索客户数据的请求，红色箭头显示向上流动到屏幕以显示数据的响应。在本例中，客户信息由客户数据和订单数据（客户下的订单）组成。

客户屏幕负责接受请求并显示客户信息。它不知道数据在哪里，如何检索数据，或者必须查询多少个数据库表才能获取数据。一旦客户屏幕收到获取特定个人的客户信息的请求，它就会将该请求转发到客户代表模块。该模块负责了解业务层中的哪些模块可以处理该请求以及如何到达该模块以及它需要什么数据（合同）。业务层中的客户对象负责聚合业务请求所需的所有信息（在本例中是获取客户信息）。该模块调用持久层中的`customer dao`（数据访问对象）模块来获取客户数据，并调用`**order dao`模块来获取订单信息。这些模块依次执行SQL语句来检索相应的数据并将其传递回业务层中的客户对象。一旦客户对象接收到数据，它就会聚合数据并将该信息传递回客户代表，然后客户代表将该数据传递到客户屏幕以呈现给用户。

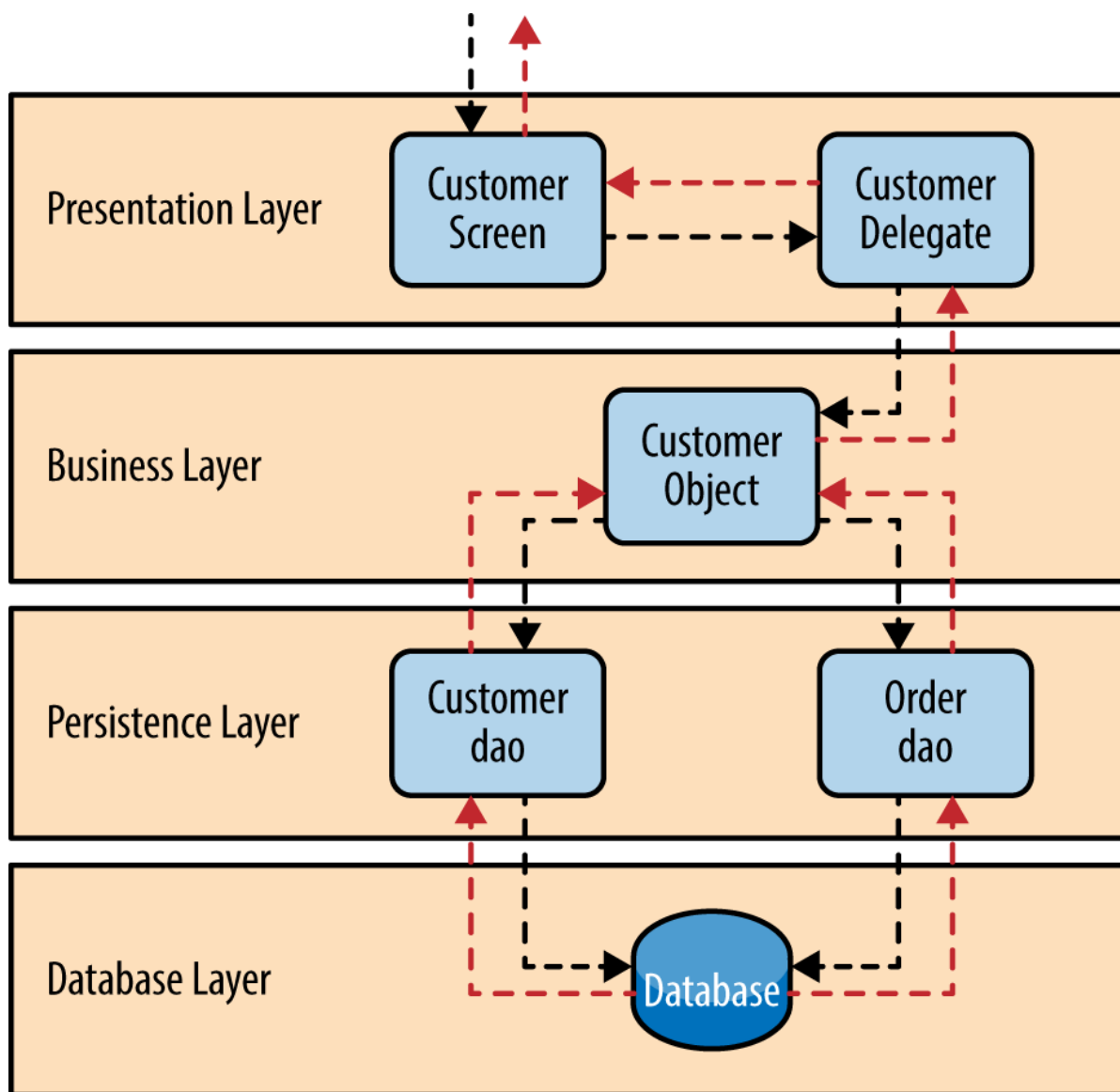


图 1-4.分层架构示例

从技术角度来看，这些模块的实现方式实际上有数十种。例如，在 Java 平台中，客户屏幕可以是 (JSF) Java Server Faces 屏幕，与作为受管 Bean 组件的客户委托耦合。业务层中的客户对象可以是本地 Spring bean 或远程 EJB3 bean。前面示例中说明的数据访问对象可以实现为简单的 POJO（普通 Java 对象）、MyBatis XML Mapper 文件，甚至封装原始 JDBC 调用或 Hibernate 查询的对象。从微软平台的角度来看，客户屏幕可以是一个 ASP（活动服务器页面）模块，使用 .NET 框架访问业务层的 C# 模块，客户和订单数据访问模块实现为 ADO（ActiveX 数据对象）。

注意事项

分层架构模式是一种可靠的通用模式，使其成为大多数应用程序的良好起点，特别是当您不确定哪种架构模式最适合您的应用程序时。然而，在选择这种模式时，从架构的角度来看，有一些事情需要考虑。

首先要注意的是所谓的架构污水坑反模式。这种反模式描述了请求流经架构的多个层的情况，作为简单的直通处理，每层内执行很少或不执行逻辑。例如，假设表示层响应用户检索客户数据的请求。表示层将请求传递给业务层，业务层将请求简单地传递给持久层，然后持久层对数据库层进行简单的 SQL 调用以检索客户数据。然后，数据会一路传回堆栈，无需额外的处理或逻辑来聚合、计算或转换数据。

每个分层架构都会至少有一些落入架构污水坑反模式的场景。然而，关键是分析属于此类别的请求的百分比。80-20 规则通常是一个很好的实践，可以用来确定您是否正在经历架构污水坑反模式。通常，大约 20% 的请求是简单的直通处理，而 80% 的请求具有与该请求关联的某些业务逻辑。然而，如果您发现这个比例是相反的，并且您的大多数请求都是简单的直通处理，您可能需要考虑开放一些架构层，请记住，由于以下原因，控制更改将变得更加困难缺乏层隔离。

分层架构模式的另一个考虑因素是，即使您将表示层和业务层拆分为单独的可部署单元，它也倾向于采用整体应用程序。虽然这对于某些应用程序来说可能不是问题，但它确实部署、总体稳健性和可靠性、性能和可扩展性方面提出了一些潜在问题。

模式分析

下表包含对分层架构模式的常见架构特征的评级和分析。每个特征的评级基于该特征作为基于模式的典型实现的能力的自然趋势，以及该模式通常为人所知的内容。有关此模式与本报告中其他模式的关系的并排比较，请参阅 本报告末尾的[模式分析摘要](#)。

- **整体敏捷性**

评级：低

分析：整体敏捷性是指对不断变化的环境做出快速响应的能力。虽然可以通过此模式的隔离功能层来隔离更改，但由于大多数实现的整体性质以及通常与此相关的组件的紧密耦合，因此在此架构模式中进行更改仍然很麻烦且耗时。图案。

- **易于部署**

评级：低

分析：根据您实现此模式的方式，部署可能会成为一个问题，特别是对于大型应用程序。对组件的一个小更改可能需要重新部署整个应用程序（或应用程序的很大一部分），从而导致部署需要在非工作时间或周末进行规划、安排和执行。因此，这种模式不容易适合持续交付管道，从而进一步降低了部署的总体评级。

- **可测试性**

评级：高

分析：由于组件属于架构中的特定层，因此可以模拟或存根其他层，使得该模式相对容易测试。开发人员可以模拟演示组件或屏幕以隔离业务组件内的测试，也可以模拟业务层以测试某些屏幕功能。

- **表现**

评级：低

分析：虽然某些分层架构确实可以很好地执行，但该模式并不适合高性能应用程序，因为必须经过多个架构层才能满足业务请求，效率低下。

- **可扩展性**

评级：低

分析：由于此模式的紧密耦合和整体实现的趋势，使用此架构模式构建的应用程序通常难以扩展。您可以通过将层拆分为单独的物理部署或将整个应用程序复制到多个节点来扩展分层架构，但总体粒度太宽，导致扩展成本高昂。

- **易于开发**

评级：高

分析：易于开发获得相对较高的分数，主要是因为这种模式众所周知并且实现起来不太复杂。由于大多数公司通过按层（表示、业务、数据库）分离技能集来开发应用程序，因此这种模式成为大多数业务应用程序开发的自然选择。公司的沟通和组织结构与其开发软件的方式之间的联系被概述为所谓的 *康威定律*。您可以通过谷歌搜索“康威定律”来获取有关这种令人着迷的相关性的更多信息。

事件驱动架构

事件驱动架构模式是一种流行的分布式异步架构模式，用于生成高度可扩展的应用程序。它的适应性也很强，既可用于小型应用程序，也可用于大型复杂的应用程序。事件驱动架构由高度解耦、单一用途的事件处理组件组成，这些组件异步接收和处理事件。

事件驱动架构模式由两个主要拓扑组成：中介者和代理者。当您需要通过中央调解器协调事件中的多个步骤时，通常会使用调解器拓扑，而当您希望在不使用中央调解器的情况下将事件链接在一起时，则使用代理拓扑。由于这两种拓扑的架构特征和实现策略有所不同，因此了解每一种拓扑以了解哪一种最适合您的特定情况非常重要。

中介者拓扑

中介器拓扑对于具有多个步骤并且需要一定级别的编排来处理事件的事件非常有用。例如，进行股票交易的单个事件可能需要您首先验证交易，然后根据各种合规性规则检查该股票交易的合规性，将交易分配给经纪人，计算佣金，最后与那个经纪人。所有这些步骤都需要一定程度的编排来确定步骤的顺序以及哪些步骤可以串行和并行完成。

中介器拓扑中有四种主要类型的体系结构组件：事件队列、事件中介器、事件通道和事件处理器。事件流从客户端将事件发送到 *事件队列* 开始，事件队列用于将事件传输到事件中介器。事件 *中介器* 接收初始事件，并通过向 *事件通道* 发送附加异步事件 来执行该流程的每个步骤来编排该事件。事件 *处理器* 监听事件通道，从事件中介器接收事件并执行特定的业务逻辑来处理该事件。[图 2-1](#) 说明了事件驱动架构模式的一般中介拓扑。

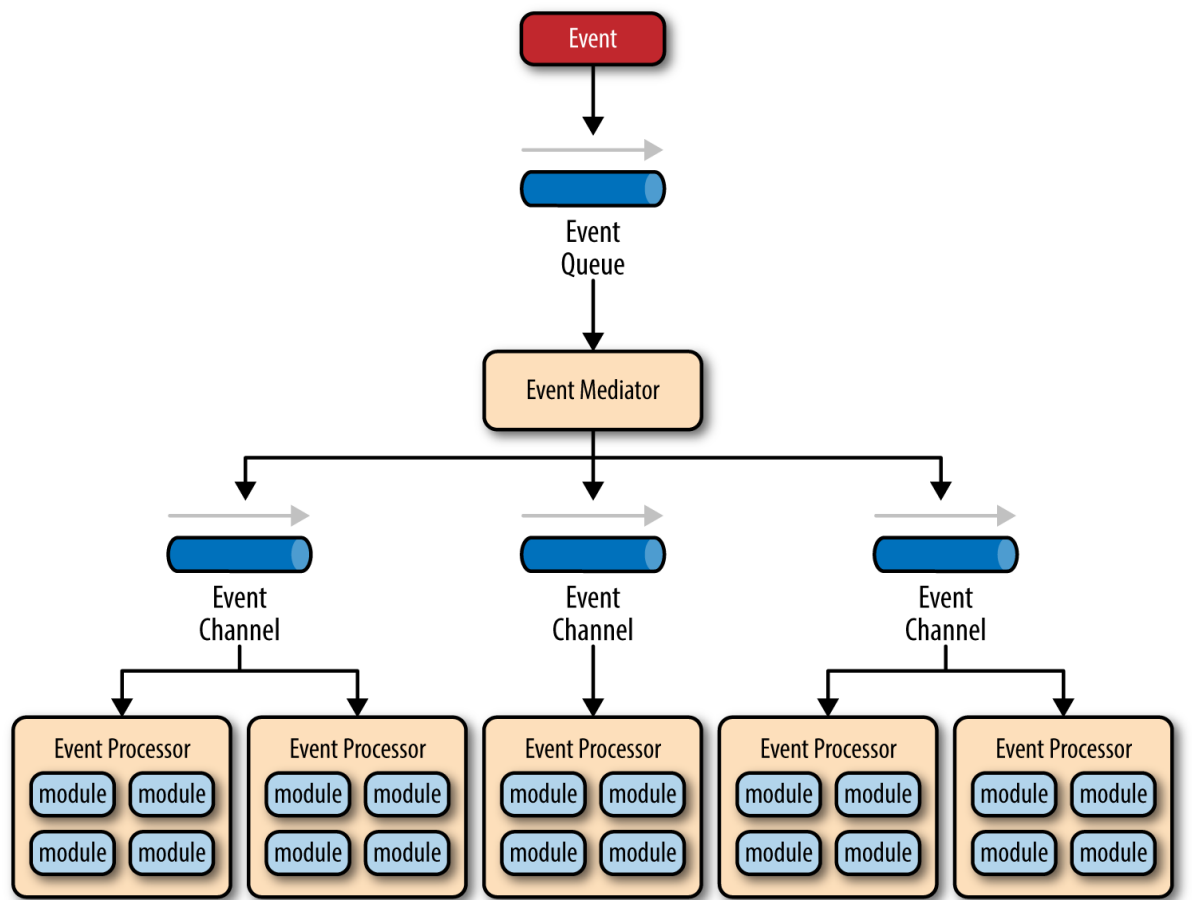


图 2-1.事件驱动架构中介拓扑

在事件驱动的体系结构中，通常有十几个到几百个事件队列。该模式没有指定事件队列组件的实现；它可以是消息队列、Web 服务端点或其任意组合。

此模式中有两种类型的事件：*初始事件*和*处理事件*。初始事件是中介器接收到的原始事件，而处理事件是由中介器生成并由事件处理组件接收的事件。

事件中介组件负责编排初始事件中包含的步骤。对于初始事件中的每个步骤，事件中介器将特定的处理事件发送到事件通道，然后由事件处理器接收并处理。需要注意的是，事件中介器实际上并不执行处理初始事件所需的业务逻辑；而是执行处理初始事件所需的业务逻辑。相反，它知道处理初始事件所需的步骤。

事件中介器使用事件通道将与初始事件中每个步骤相关的特定处理事件异步传递到事件处理器。事件通道可以是消息队列或消息主题，尽管消息主题最广泛地与中介器拓扑一起使用，以便处理事件可以由多个事件处理器处理（每个事件处理器根据接收到的处理事件执行不同的任务）。

事件处理器组件包含处理处理事件所需的应用程序业务逻辑。事件处理器是自包含、独立、高度解耦的架构组件，用于执行应用程序或系统中的特定任务。虽然事件处理器组件的粒度可以从细粒度（例如，计算订单的销售税）到粗粒度（例如，处理保险索赔）不等，但重要的是要记住，一般来说，每个事件处理器组件应该执行单个业务任务，而不是依赖其他事件处理器来完成其特定任务。

事件中介器可以通过多种方式实现。作为架构师，您应该了解每个实施选项，以确保您为事件中介器选择的解决方案符合您的需求和要求。

事件中介器最简单且最常见的实现是通过开源集成中心，例如 Spring Integration、Apache Camel 或 Mule ESB。这些开源集成中心中的事件流通常通过 Java 代码或 DSL（特定于领域的语言）来实现。对于更复杂的中介和编排，您可以将 BPEL（业务流程执行语言）与 BPEL 引擎（例如开源 Apache ODE）结合使用。BPEL 是一种标准的类似 XML 的语言，它描述了处理初始事件所需的数据和步骤。对于需要更复杂的编排（包括涉及人工交互的步骤）的大型应用程序，您可以使用业务流程管理器（BPM）（例如 jBPM）来实现事件中介器。

了解您的需求并将其与正确的事件中介器实现相匹配对于使用此拓扑的任何事件驱动架构的成功至关重要。使用开源集成中心进行非常复杂的业务流程管理编排是失败的秘诀，就像实施 BPM 解决方案来执行简单的路由逻辑一样。

为了说明中介拓扑的工作原理，假设您通过保险公司投保并且决定搬家。在这种情况下，初始事件可能称为“*重定位事件*”。处理*重定位事件*所涉及的步骤包含在事件中介器中，如图2-2所示。对于每个初始事件步骤，事件中介器创建一个处理事件（例如，*更改地址*、*重新计算报价*等），将该处理事件发送到事件通道，并等待相应事件处理器处理该处理事件（例如、客户流程、报价流程等）。此过程将持续进行，直到初始事件中的所有步骤均已处理完毕。事件中介器中重新计算报价和更新索赔步骤上方的单条表示这些步骤可以同时运行。

代理拓扑

代理拓扑与中介拓扑的不同之处在于，没有中央事件中介。相反，消息流通过轻量级消息代理（例如，ActiveMQ、HornetQ等）以链式方式分布在事件处理器组件上。当您有相对简单的事件处理流程并且不需要（或不需要）中央事件编排时，此拓扑非常有用。

代理拓扑中有两种主要类型的架构组件：*代理组件*和*事件处理器组件*。代理组件可以是集中式的或联合式的，并且包含事件流中使用的所有事件通道。代理组件中包含的事件通道可以是消息队列、消息主题或两者的组合。

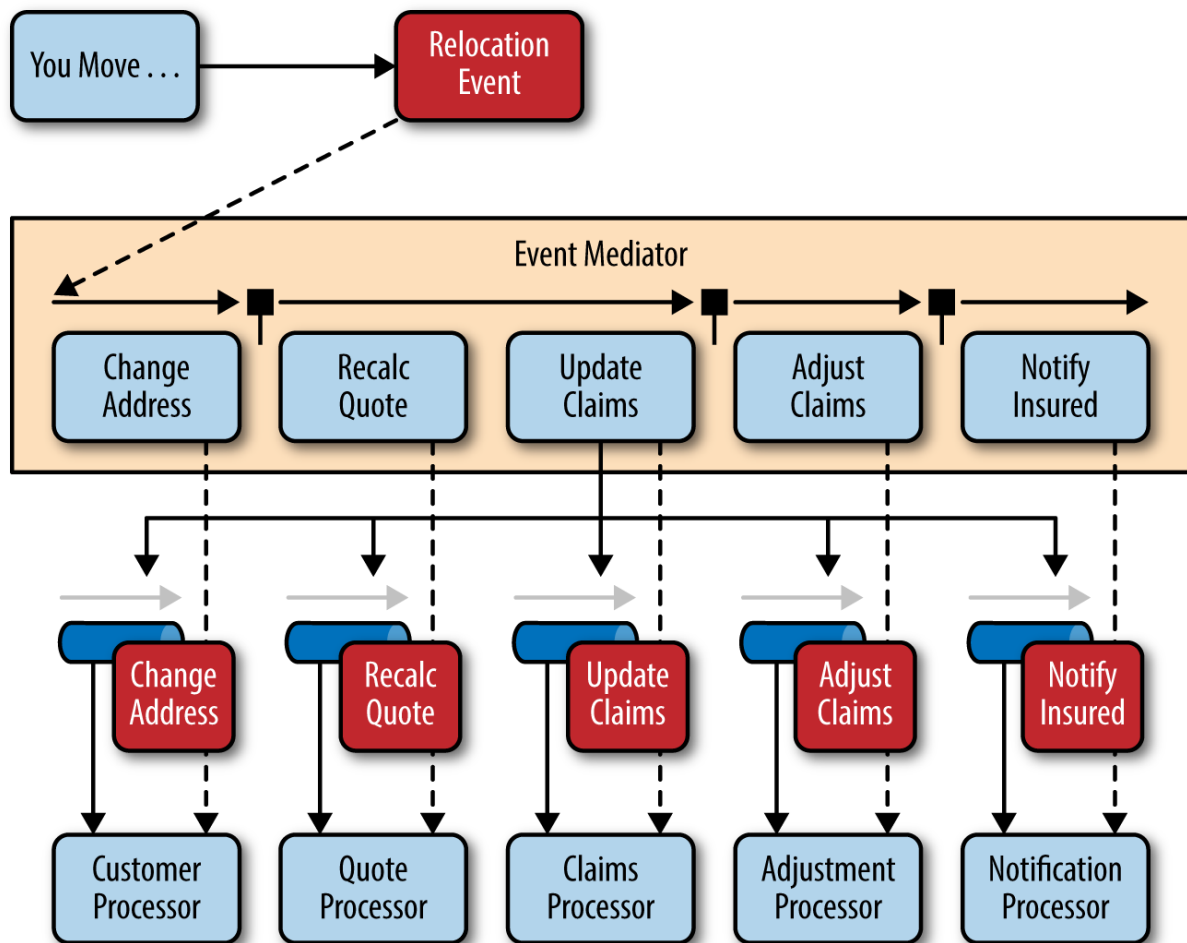


图 2-2.中介者拓扑示例

该拓扑如图2-3所示。从图中可以看出，没有中央事件中介组件来控制 and 编排初始事件；相反，每个事件处理器组件负责处理事件并发布一个新事件来指示它刚刚执行的操作。例如，平衡股票投资组合的事件处理器可能会收到称为股票分割的初始事件。基于该初始事件，事件处理器可能会进行一些投资组合重新平衡，然后向经纪商发布一个名为“重新平衡投资组合”的新事件，然后该事件将由不同的事件处理器拾取。请注意，有时事件处理器可能会发布事件，但不会被任何其他事件处理器拾取。当您开发应用程序或提供未来的功能和扩展时，这种情况很常见。

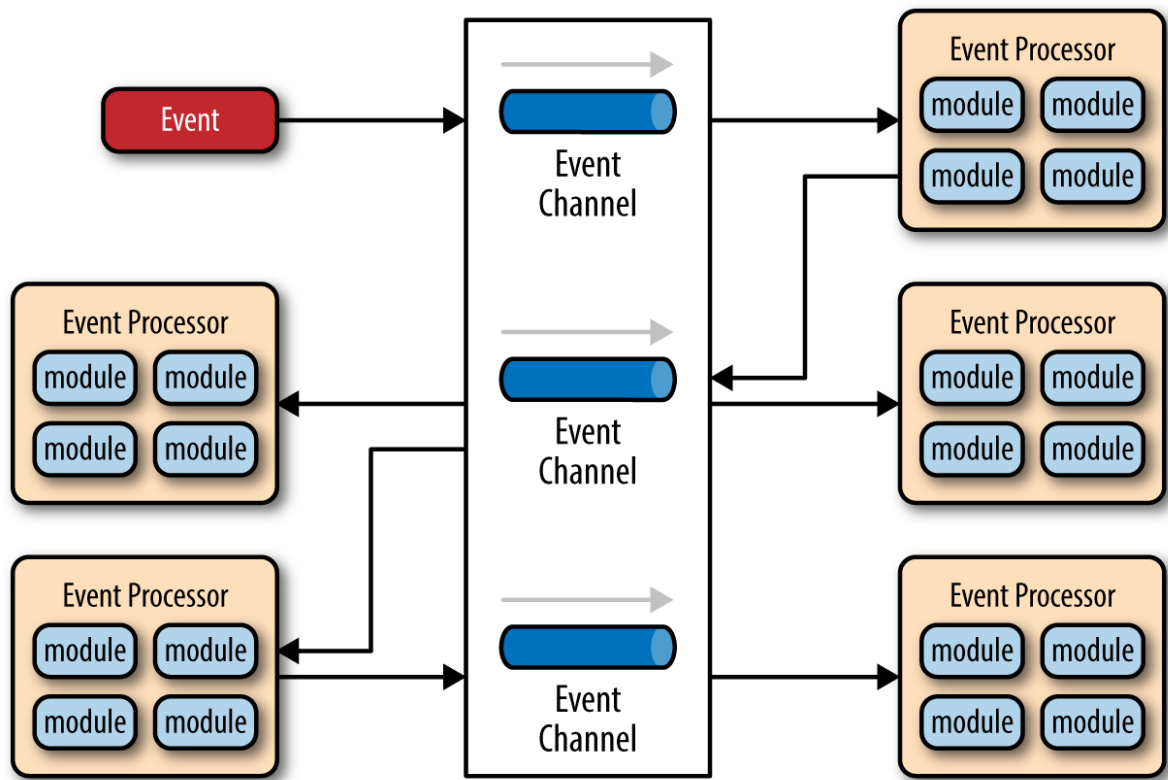


图 2-3.事件驱动架构代理拓扑

为了说明经纪人拓扑的工作原理，我们将使用与中介拓扑相同的示例（被保险人移动）。由于代理拓扑中没有中央事件中介来接收初始事件，因此客户进程组件直接接收事件，更改客户地址，并发出一个事件，表示它更改了客户地址（例如，*更改地址* 事件）。在此示例中，有两个对*更改地址*事件感兴趣的事件处理器：报价流程和索赔流程。报价处理器组件根据地址更改重新计算新的自动保险费率，并向系统的其余部分发布一个事件，指示其执行的操作（例如，*重新计算报价*事件）。另一方面，索赔处理组件接收相同的*更改地址*事件，但在这种情况下，它更新未决的保险索赔并将事件作为**更新索赔事件发布到系统。然后，其他事件处理器组件会拾取这些新事件，事件链将继续通过系统，直到不再有针对该特定启动事件的事件发布。

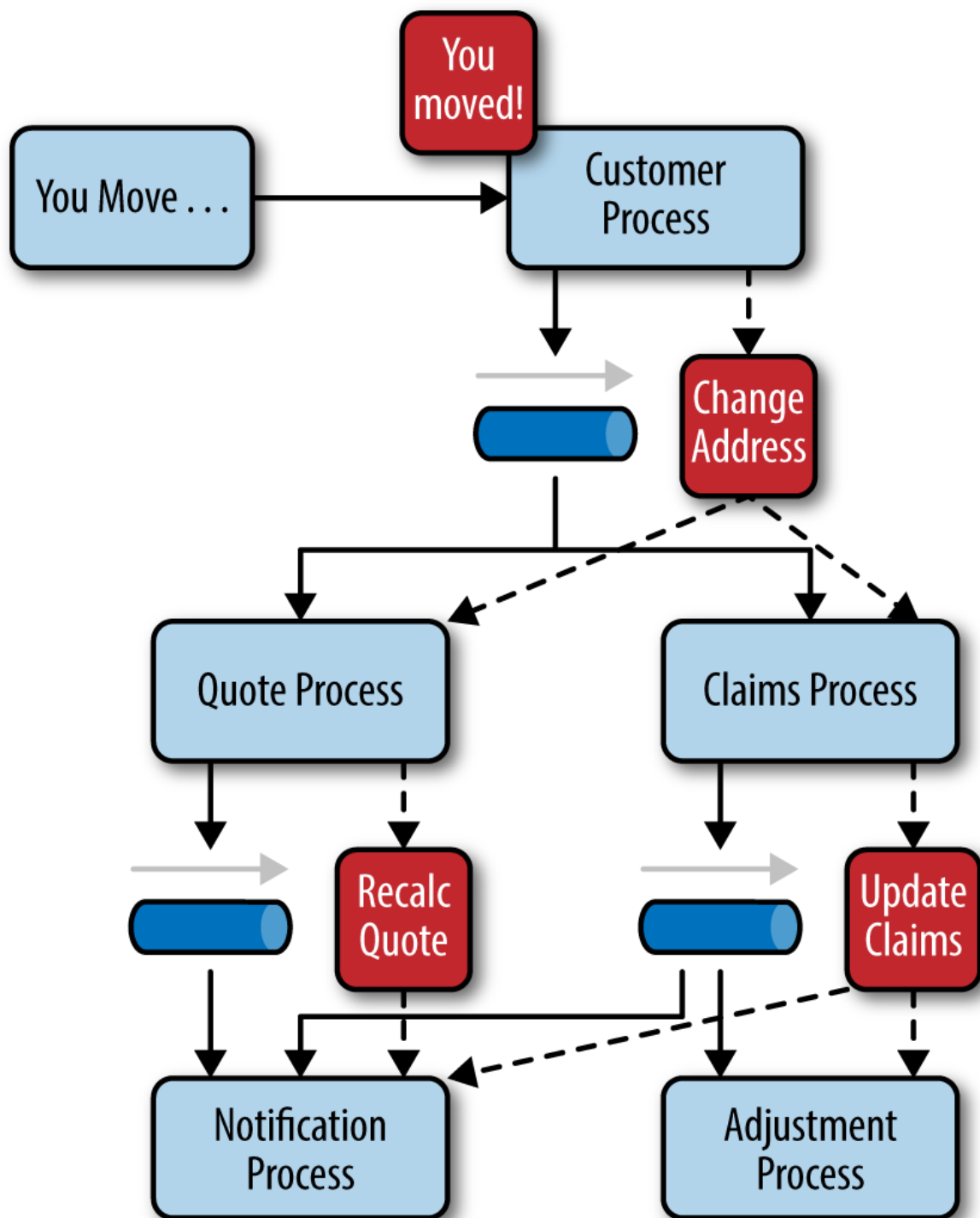


图 2-4.代理拓扑示例

从图 2-4 中可以看出，代理拓扑就是通过事件链来执行业务功能。了解代理拓扑的最佳方法是将其视为接力赛。在接力赛中，赛跑者手持接力棒跑一段距离，然后将接力棒交给下一个赛跑者，依此类推，直到最后一个赛跑者冲过终点线。在接力赛中，一旦赛跑者交出接力棒，她就完成了比赛。对于代理拓扑也是如此：一旦事件处理器移交事件，它就不再参与该特定事件的处理。

注意事项

事件驱动架构模式是一种实现起来相对复杂的模式，主要是由于其异步分布式特性。实现此模式时，您必须解决各种分布式架构问题，例如远程进程可用性、缺乏响应能力以及代理或中介器故障时的代理重新连接逻辑。

选择此架构模式时需要考虑的一个因素是单个业务流程缺乏原子事务。由于事件处理器组件是高度解耦和分布式的，因此很难在它们之间维护事务工作单元。因此，在使用此模式设计应用程序时，您必须不断考虑哪些事件可以独立运行，哪些事件不能独立运行，并相应地规划事件处理器的粒度。如果您发现需要跨事件处理器拆分单个工作单元（也就是说，如果您将单独的处理器用于应该是不可分割事务的事务），那么这可能不是适合您的应用程序的模式。

也许事件驱动架构模式最困难的方面之一是事件处理器组件契约的创建、维护和治理。每个事件通常都有与其关联的特定契约（例如，传递到事件处理器的数据值和数据格式）。使用此模式时，确定标准数据格式（例如 XML、JSON、Java 对象等）并从一开始就建立合同版本控制策略至关重要。

模式分析

下表包含对事件驱动架构模式的常见架构特征的评级和分析。每个特征的评级基于该特征作为基于模式的典型实现的能力的自然趋势，以及该模式通常为人所知的内容。有关此模式与本报告中其他模式的关系的并排比较，请参阅 本报告末尾的[模式分析摘要](#)。

- **整体敏捷性**

评级：高

分析：整体敏捷性是指对不断变化的环境做出快速响应的能力。由于事件处理器组件是单一用途的，并且与其他事件处理器组件完全解耦，因此更改通常会隔离到一个或几个事件处理器，并且可以快速进行，而不会影响其他组件。

- **易于部署**

评级：高

分析：总体而言，由于事件处理器组件的解耦性质，该模式相对容易部署。代理拓扑往往比中介器拓扑更容易部署，主要是因为事件中介器组件在某种程度上与事件处理器紧密耦合：事件处理器组件的更改可能也需要事件中介器的更改，从而要求两者都必须更改。为任何给定的更改进行部署。

- **可测试性**

评级：低

分析：虽然单个单元测试并不太困难，但它确实需要某种专门的测试客户端或测试工具来生成事件。这种模式的异步特性也使测试变得复杂。

- **表现**

评级：高

分析：虽然确实有可能实现一个事件驱动的架构，但由于涉及到所有消息传递基础设施，该架构的性能不佳，但一般来说，该模式通过其异步功能实现了高性能；换句话说，执行解耦、并行异步操作的能力超过了消息排队和出队的成本。

- **可扩展性**

评级：高

分析：在这种模式中，可扩展性是通过高度独立和解耦的事件处理器自然实现的。每个事件处理器都可以单独扩展，从而实现细粒度的可扩展性。

- **易于开发**

评级：低

分析：由于模式和合约创建的异步性质，以及代码中需要更高级的错误处理条件来应对无响应的事件处理器和失败的代理，开发可能会有些复杂。

微内核架构

微内核架构模式（有时称为插件架构模式）是实现基于产品的应用程序的自然模式。基于产品的应用程序是作为典型第三方产品打包并提供版本下载的应用程序。然而，许多公司还开发和发布其内部业务应用程序，例如软件产品，包含版本、发行说明和可插入功能。这些也非常适合这种模式。微内核架构模式允许您将额外的应用程序功能作为插件添加到核心应用程序中，从而提供可扩展性以及功能分离和隔离。

模式描述

微内核架构模式由两类架构组件组成：*核心系统*和*插件模块*。应用逻辑划分为独立的插件模块和基础核心系统，提供可扩展性、灵活性以及应用功能和自定义处理逻辑的隔离。[图 3-1](#) 说明了基本的微内核架构模式。

微内核架构模式的核心系统传统上只包含系统运行所需的最少功能。许多操作系统都实现微内核架构模式，因此该模式名称的由来。从业务应用程序的角度来看，核心系统通常被定义为通用业务逻辑，没有针对特殊情况、特殊规则或复杂条件处理的自定义代码。

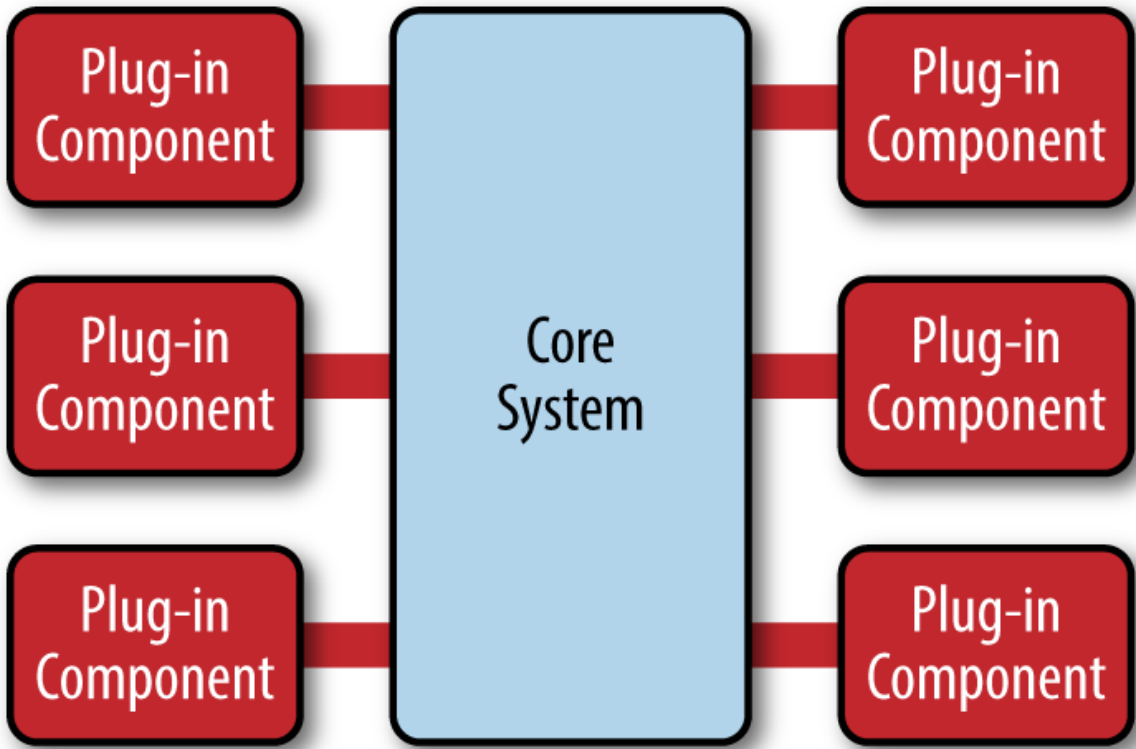


图 3-1.微内核架构模式

插件模块是独立的组件，包含专门的处理、附加功能和自定义代码，旨在增强或扩展核心系统以产生附加业务功能。一般来说，插件模块应该独立于其他插件模块，但是您当然可以设计需要其他插件存在的插件。无论哪种方式，将插件之间的通信保持在最低限度以避免依赖性问题都很重要。

核心系统需要知道哪些插件模块可用以及如何访问它们。实现此目的的一种常见方法是通过某种插件注册表。该注册表包含有关每个插件模块的信息，包括其名称、数据协定和远程访问协议详细信息（取决于插件如何连接到核心系统）。例如，标记高风险税务审计项目的税务软件插件可能有一个注册表项，其中包含服务名称(AuditChecker)、数据合同（输入数据和输出数据）以及合同格式（XML）。如果通过 SOAP 访问插件，它还可能包含 WSDL（Web 服务定义语言）。

插件模块可以通过多种方式连接到核心系统，包括OSGi（开放服务网关倡议）、消息传递、Web服务，甚至直接点对点绑定（即对象实例化）。您使用的连接类型取决于您正在构建的应用程序类型（小型产品或大型业务应用程序）以及您的特定需求（例如，单一部署或分布式部署）。架构模式本身没有指定任何这些实现细节，只是插件模块必须保持彼此独立。

插件模块和核心系统之间的合同范围可以从标准合同到定制合同。自定义合约通常出现在插件组件由第三方开发的情况下，您无法控制插件使用的合约。在这种情况下，通常在插件联系人和标准合约之间创建一个适配器，以便核心系统不需要为每个插件编写专门的代码。创建标准合约（通常通过 XML 或 Java 映射实现）时，重要的是要记住从一开始就创建版本控制策略。

模式示例

也许微内核架构的最佳示例是 Eclipse IDE。下载基本的 Eclipse 产品为您提供的只不过是一个精美的编辑器。然而，一旦您开始添加插件，它就会成为一个高度可定制且有用的产品。互联网浏览器是使用微内核架构的另一个常见产品示例：查看器和其他插件添加了基本浏览器（即核心系统）中没有的附加功能。

对于基于产品的软件来说，例子是无穷无尽的，但是大型商业应用程序呢？微内核架构也适用于这些情况。为了说明这一点，让我们使用另一家保险公司的例子，但这一次涉及保险索赔处理。

索赔处理是一个非常复杂的过程。每个州对于保险索赔中允许的内容和不允许的内容都有不同的规则和规定。例如，如果您的挡风玻璃被岩石损坏，某些州允许免费更换挡风玻璃，而其他州则不允许。这为标准索赔流程创造了几乎无限的条件。

毫不奇怪，大多数保险索赔应用程序都利用大型且复杂的规则引擎来处理大部分复杂性。然而，这些规则引擎可能会成长为一个复杂的大泥球，其中更改一个规则会影响其他规则，或者进行简单的规则更改需要一支分析师、开发人员和测试人员队伍。使用微内核架构模式可以解决其中许多问题。

您在图 3-2 中看到的文件夹堆栈代表了索赔处理的核心系统。它包含保险公司处理索赔所需的基本业务逻辑，但没有任何自定义处理。每个插件模块都包含该状态的特定规则。在此示例中，可以使用自定义源代码或单独的规则引擎实例来实现插件模块。无论实现如何，关键点是特定于状态的规则和处理与核心索赔系统分开，并且可以添加、删除和更改，而对核心系统的其余部分或其他插件模块影响很小或没有影响。

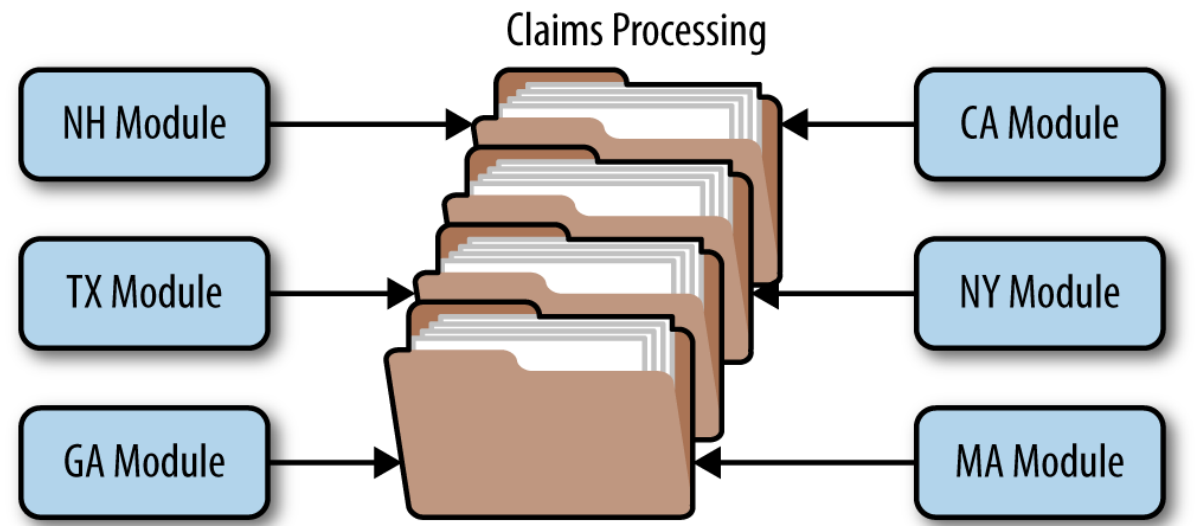


图 3-2.微内核架构示例

注意事项

微内核架构模式的一个伟大之处在于它可以嵌入或用作另一个架构模式的一部分。例如，如果此模式解决了应用程序的特定易失性区域所遇到的特定问题，您可能会发现无法使用此模式实现整个体系结构。在这种情况下，您可以将微服务架构模式嵌入到您正在使用的另一种模式中（例如，分层架构）。同样，上一节中有关事件驱动架构的事件处理器组件可以使用微服务架构模式来实现。

微服务架构模式为演化设计和增量开发提供了强大的支持。您可以首先生成一个可靠的核心系统，然后随着应用程序的逐步发展，添加特性和功能，而无需对核心系统进行重大更改。

对于基于产品的应用程序，微内核架构模式应该始终是您作为起始架构的首选，特别是对于那些您将随着时间的推移发布附加功能并希望控制哪些用户获得哪些功能的产品。如果随着时间的推移，您发现该模式不能满足您的所有要求，您始终可以将应用程序重构为更适合您的特定要求的另一种架构模式。

模式分析

下表包含对微内核架构模式的常见架构特征的评级和分析。每个特征的评级基于该特征作为基于模式的典型实现的能力的自然趋势，以及该模式通常为人所知的内容。有关此模式与本报告其他模式的关系的并排比较，请参阅本报告末尾的[模式分析摘要](#)。

- **整体敏捷性**

评级：高

分析：整体敏捷性是指对不断变化的环境做出快速响应的能力。通过松散耦合的插件模块，可以在很大程度上隔离并快速实施更改。一般来说，大多数微内核架构的核心系统往往会很快变得稳定，因此相当健壮，并且随着时间的推移几乎不需要改变。

- **易于部署**

评级：高

分析：根据模式的实现方式，插件模块可以在运行时动态添加到核心系统（例如热部署），从而最大限度地减少部署期间的停机时间。

- **可测试性**

评级：高

分析：插件模块可以单独测试，并且可以轻松地从核心系统模拟，以演示或原型化特定功能，而无需对核心系统进行很少的更改或无需更改。

- **表现**

评级：高

分析：虽然微内核模式本身并不适合高性能应用程序，但一般来说，使用微内核架构模式构建的大多数应用程序都表现良好，因为您可以自定义和简化应用程序以仅包含您需要的功能。JBoss 应用程序服务器就是一个很好的例子：通过其插件架构，您可以将应用程序服务器精简为仅包含您需要的功能，从而删除昂贵的未使用功能，例如消耗内存的远程访问、消息传递和缓存、CPU 和线程，并降低应用程序服务器的速度。

- **可扩展性**

评级：低

分析：由于大多数微内核架构实现都是基于产品的，并且尺寸通常较小，因此它们作为单个单元实现的，因此不具备高度可扩展性。根据您实现插件模块的方式，您有时可以在插件功能级别提供可扩展性，但总体而言，这种模式并不适合生成高度可扩展的应用程序。

- **易于开发**

评级：低

分析：微内核架构需要深思熟虑的设计和合约治理，使其实现起来相当复杂。合同版本控制、内部插件注册表、插件粒度以及可用于插件连接的广泛选择都增加了实现此模式所涉及的复杂性。

微服务架构模式

微服务架构模式作为整体应用程序和面向服务的架构的可行替代方案，正在业界迅速普及。由于这种架构模式仍在不断发展，因此业界对于这种模式的含义及其实现方式存在很多困惑。报告的这一部分将为您提供了解这一重要架构模式的好处（和权衡）以及它是否适合您的应用程序所需的关键概念和基础知识。

模式描述

无论您选择哪种拓扑或实现风格，都有几个适用于通用架构模式的通用核心概念。第一个概念是*单独部署的单元*的概念。如图4-1所示，微服务架构的每个组件都部署为单独的单元，从而可以通过有效且简化的交付管道、更高的可扩展性以及应用程序内的应用程序和组件的高度解耦来更轻松地进行部署。

对于此模式，需要理解的最重要的概念也许是*服务组件*的概念。与其考虑微服务架构中的服务，不如考虑服务组件，服务组件的粒度可能会有所不同，从单个模块到应用程序的大部分。服务组件包含一个或多个模块（例如，Java 类），这些模块代表单一用途的功能（例如，提供特定城市或城镇的天气）或大型业务应用程序的独立部分（例如，股票交易布局或确定汽车保险费率）。设计正确级别的服务组件粒度是微服务架构中最大的挑战之一。下面的服务组件编排小节将更详细地讨论这一挑战。

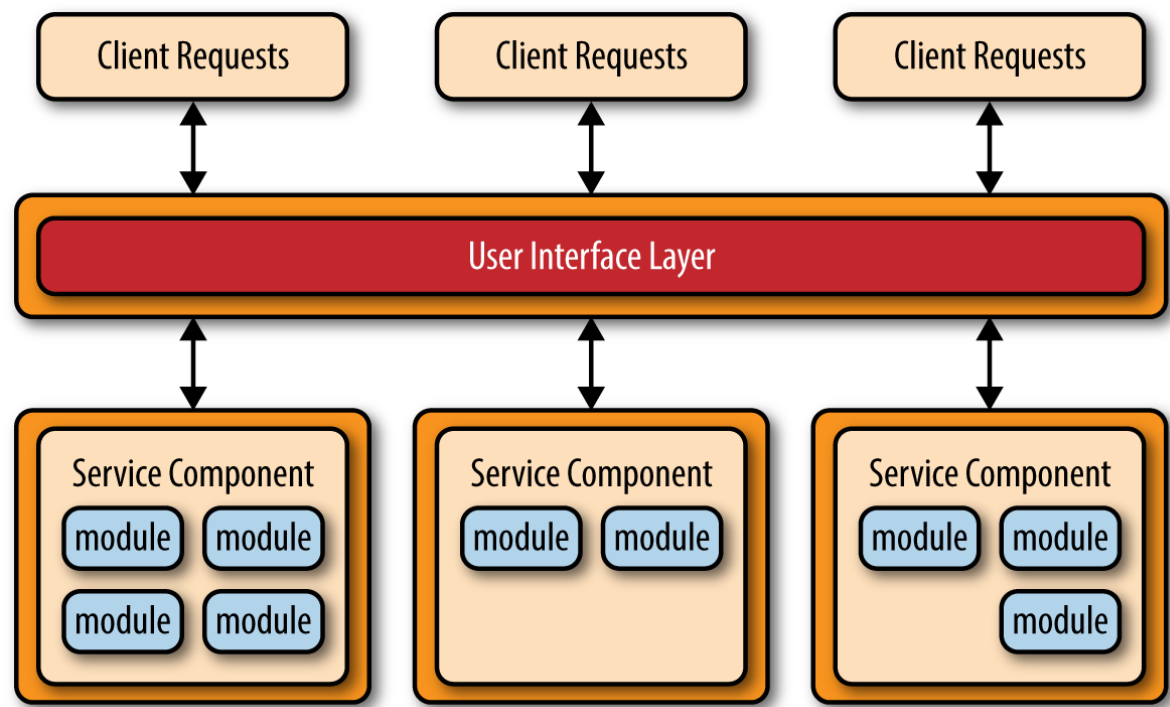


图 4-1. 基本微服务架构模式

微服务架构模式中的另一个关键概念是它是一种分布式架构，这意味着该架构中的所有组件都彼此完全解耦，并通过某种远程访问协议（例如，JMS、AMQP、REST、SOAP、RMI 等）。这种架构模式的分布式本质是它如何实现一些卓越的可扩展性和部署特性。

微服务架构令人兴奋的事情之一是，它是从与其他常见架构模式相关的问题演变而来的，而不是作为等待问题发生的解决方案而创建的。微服务架构风格自然地两个主要来源演变而来：使用分层架构模式开发的单体应用程序和通过面向服务的架构模式开发的分布式应用程序。

从整体应用程序到微服务架构风格的演进路径主要是通过持续交付的发展来推动的，持续交付是从开发到生产的持续部署管道的概念，它简化了应用程序的部署。单体应用程序通常由紧密耦合的组件组成，这些组件是单个可部署单元的一部分，这使得更改、测试和部署应用程序变得繁琐且困难（因此，大多数大型 IT 商店中常见的“每月部署”周期兴起））。这些因素通常会导致应用程序脆弱，每次部署新内容时就会崩溃。微服务架构模式通过将应用程序分为多个可部署单元（服务组件）来解决这些问题，这些单元可以独立于其他服务组件单独开发、测试和部署。

导致微服务架构模式的另一个演进路径来自于实现面向服务的架构模式 (SOA) 的应用程序中发现的问题。尽管 SOA 模式非常强大，并且提供了无与伦比的抽象级别、异构连接、服务编排以及将业务目标与 IT 功能保持一致的承诺，但它仍然复杂、昂贵、普遍、难以理解和实现，并且通常是杀伤力过大的。对于大多数应用程序。微服务架构风格通过简化服务概念、消除编排需求以及简化对服务组件的连接和访问来解决这种复杂性。

模式拓朴

虽然实现微服务架构模式的方法有数十种，但最常见和流行的三种主要拓朴是：基于 API REST 的拓朴、基于应用程序 REST 的拓朴和集中式消息传递拓朴。

基于 API REST 的拓朴对于通过某种 API（应用程序编程接口）公开小型、独立的单独服务的网站非常有用。这种拓朴如图 4-2 所示，由非常细粒度的服务组件（因此称为微服务）组成，其中包含一两个独立于其余服务执行特定业务功能的模块。在此拓朴中，通常使用通过单独部署的基于 Web 的 API 层实现的基于 REST 的接口来访问这些细粒度服务组件。此拓朴的示例包括 Yahoo、Google 和 Amazon 发现的一些常见的单一用途的基于云的 RESTful Web 服务。

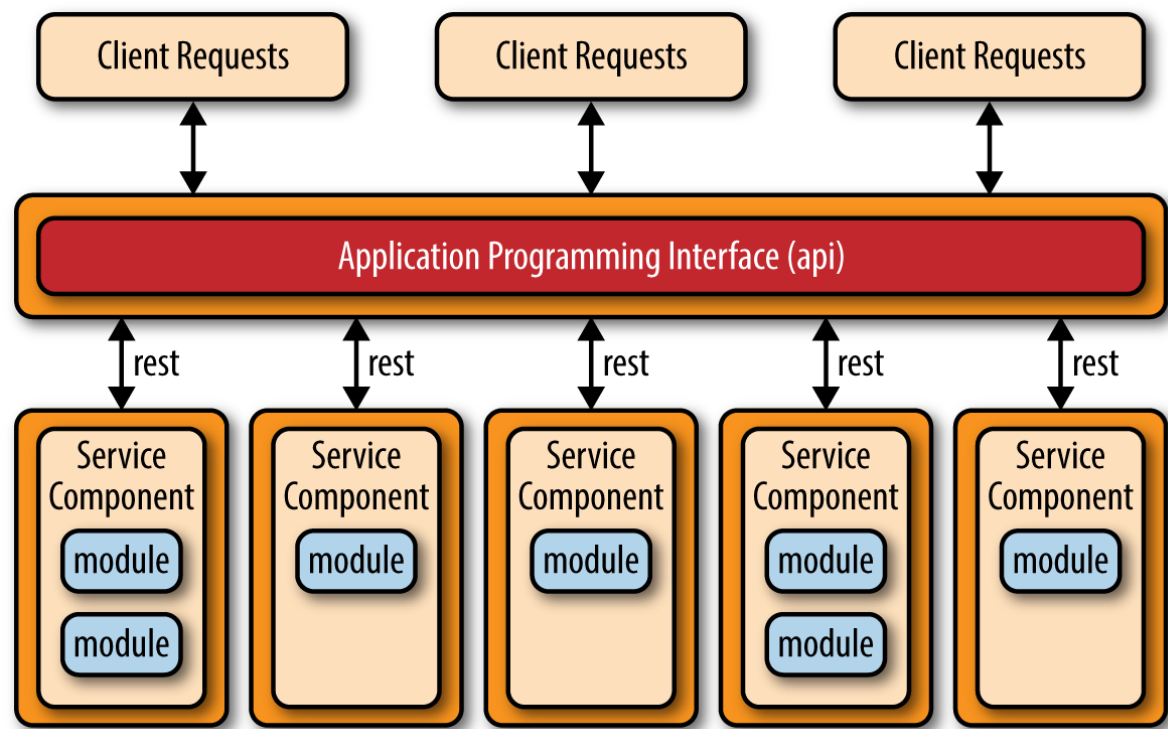


图 4-2.基于 API REST 的拓朴

基于 REST 的应用程序拓朴与基于 API REST 的方法的不同之处在于，客户端请求是通过传统的基于 Web 或胖客户端业务应用程序屏幕而不是通过简单的 API 层接收的。如图 4-3 所示，应用程序的用户界面层部署为单独的 Web 应用程序，通过简单的基于 REST 的接口远程访问单独部署的服务组件（业务功能）。此拓朴中的服务组件与基于 API-REST 的拓朴中的服务组件不同，因为这些服务组件往往更大、更粗粒度，并且代表整个业务应用程序的一小部分，而不是细粒度的、单一的。行动服务。此拓朴对于复杂性相对较低的中小型业务应用程序很常见。

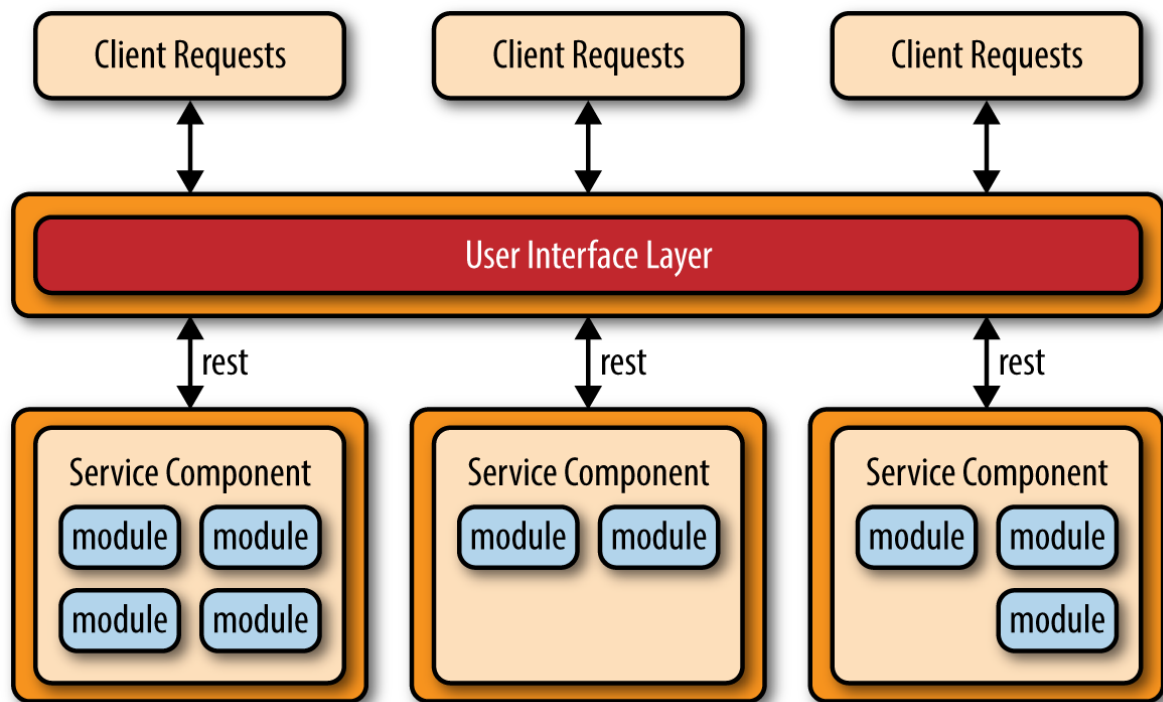


图 4-3.基于 REST 的应用程序拓扑

微服务架构模式中的另一种常见方法是集中式消息传递拓扑。该拓扑（如图4-4所示）与之前的基于REST的拓扑类似，只不过该拓扑不使用REST进行远程访问，而是使用轻量级的集中式消息代理（例如ActiveMQ、HornetQ等）。在查看此拓扑时，不要将其与面向服务的架构模式混淆或将其视为“SOA-Lite”，这一点至关重要。此拓扑中的轻量级消息代理不执行任何编排、转换或复杂的路由；相反，它只是访问远程服务组件的轻量级传输。

集中式消息传递拓扑通常出现在较大的业务应用程序或需要对用户界面和服务组件之间的传输层进行更复杂控制的应用程序中。与前面讨论的基于 REST 的简单拓扑相比，此拓扑的优点是先进的排队机制、异步消息传递、监视、错误处理以及更好的整体负载平衡和可扩展性。通常与集中式代理相关的单点故障和架构瓶颈问题可以通过代理集群和代理联合来解决（将单个代理实例拆分为多个代理实例，以根据系统的功能区域划分消息吞吐量负载）。

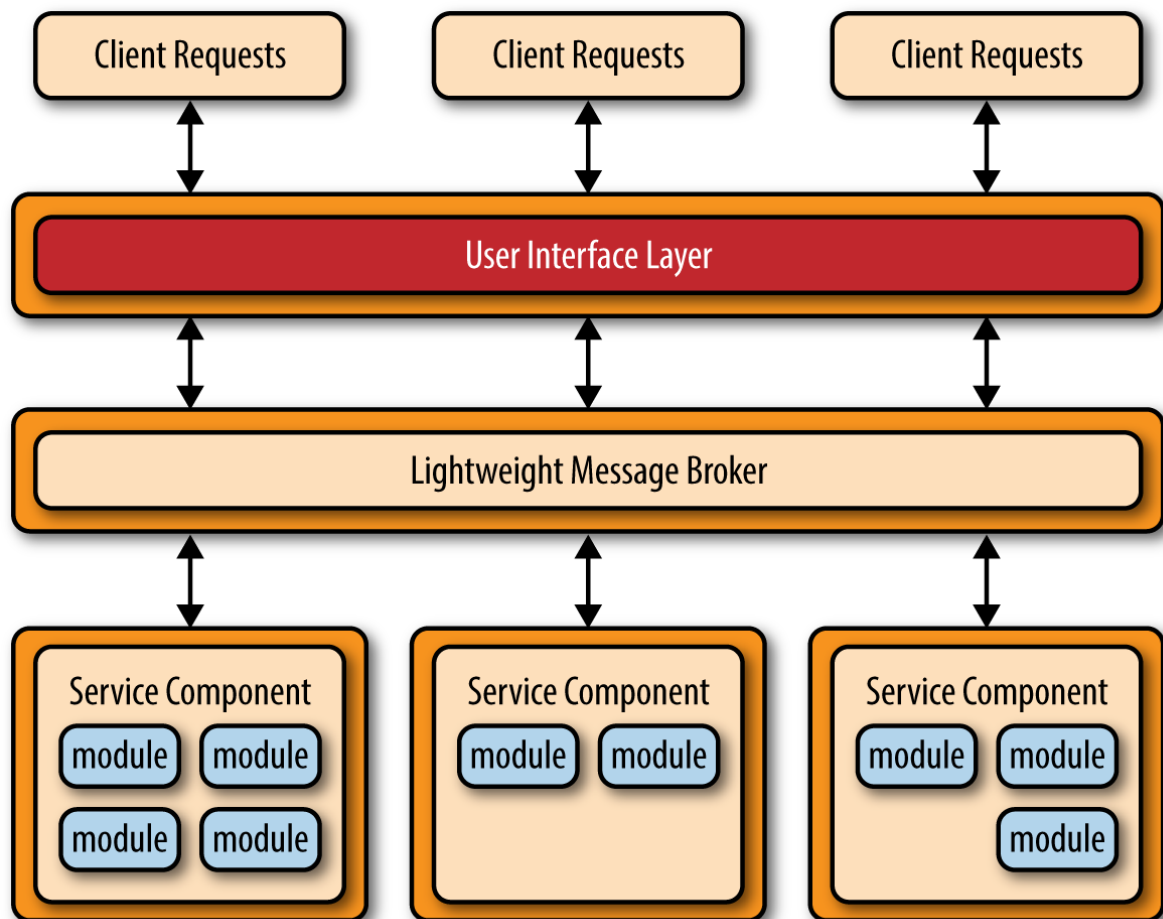


图 4-4.集中式消息传递拓扑

避免依赖和编排

微服务架构模式的主要挑战之一是确定服务组件的正确粒度级别。如果服务组件太粗粒度，您可能不会意识到这种架构模式带来的好处（部署、可扩展性、可测试性和松散耦合）。然而，太细粒度的服务组件将导致服务编排需求，这将迅速将您的精益微服务架构转变为重量级的面向服务的架构，并解决 SOA 中常见的所有复杂性、混乱、费用和缺陷。基于应用程序。

如果您发现需要从应用程序的用户界面或 API 层编排您的服务组件，那么您的服务组件很可能太细粒度了。同样，如果您发现需要在服务组件之间执行服务间通信来处理单个请求，则很可能您的服务组件要么粒度太细，要么从业务功能的角度来看它们没有正确分区。

服务间通信可能会导致组件之间产生不必要的耦合，但可以通过共享数据库来处理。例如，如果处理互联网订单的服务组件需要客户信息，它可以访问数据库来检索必要的信息，而不是调用客户服务组件内的功能。

共享数据库可以处理信息需求，但是共享功能呢？如果一个服务组件需要另一个服务组件中包含的功能或所有服务组件共有的功能，您有时可以跨服务组件复制共享的功能（从而违反了 DRY 原则：不要重复自己）。在大多数实现微服务架构模式的业务应用程序中，这是相当常见的做法，为了保持服务组件的独立性并分离其部署，需要权衡重复小部分业务逻辑的冗余。小型实用程序类可能属于此类重复代码。

如果您发现无论服务组件粒度级别如何，您仍然无法避免服务组件编排，那么这是一个好兆头，表明这可能不是适合您的应用程序的架构模式。由于此模式的分布式特性，跨服务组件（以及服务组件之间）维护单个事务工作单元非常困难。这种做法需要某种事务补偿框架来回滚事务，这给这种相对简单而优雅的架构模式增加了显著的复杂性。

注意事项

微服务架构模式解决了单体应用程序和面向服务的架构中发现的许多常见问题。由于主要应用程序组件被分割成更小的、单独部署的单元，因此使用微服务架构模式构建的应用程序通常更健壮，提供更好的可扩展性，并且可以更轻松地支持持续交付。

这种模式的另一个优点是，它提供了进行实时生产部署的能力，从而显着减少了对传统的每月或周末“大爆炸”生产部署的需求。由于变更通常隔离到特定的服务组件，因此只需要部署发生变更的服务组件。如果您只有一个服务组件实例，则可以在用户界面应用程序中编写专门的代码来检测活动的热部署并将用户重定向到错误页面或等待页面。或者，您可以在实时部署期间换入和换出服务组件的多个实例，从而在部署周期内实现连续可用性（这对于分层架构模式来说很难做到）。

最后要考虑的一个因素是，由于微服务架构模式是一种分布式架构，因此它与事件驱动架构模式中存在一些相同的复杂问题，包括合同创建、维护和管理、远程系统可用性以及远程访问认证和授权。

模式分析

下表包含对微服务架构模式的常见架构特征的评级和分析。每个特征的评级基于该特征作为基于模式的典型实现的能力的自然趋势，以及该模式通常为人所知的内容。有关此模式与本报告中其他模式的关系的并排比较，请参阅 本报告末尾的[模式分析摘要](#)。

- **整体敏捷性**

评级：高

分析：整体敏捷性是指对不断变化的环境做出快速响应的能力。由于单独部署单元的概念，更改通常隔离到各个服务组件，从而可以快速轻松地进行部署。此外，使用这种模式构建的应用程序往往是非常松散耦合的，这也有助于促进更改。

- **易于部署**

评级：高

分析：由于远程服务的细粒度和独立性，微服务模式的部署特性比率非常高。服务通常部署为单独的软件单元，从而能够在白天或晚上随时进行“热部署”。总体部署风险也显着降低，因为失败的部署能够更快地恢复，并且仅影响正在部署的服务的操作，从而导致所有其他操作的持续操作。

- **可测试性**

评级：高

分析：由于将业务功能分离和隔离到独立的应用程序中，因此可以确定测试范围，从而可以进行更有针对性的测试工作。针对特定服务组件的回归测试比针对整个整体应用程序的回归测试更容易、更可行。此外，由于此模式中的服务组件是松散耦合的，因此从开发角度来看，进行破坏应用程序另一部分的更改的机会要小得多，从而减轻了必须测试整个应用程序的一小部分的测试负担。改变。

- **表现**

评级：低

分析：虽然您可以创建从此模式实现且性能良好的应用程序，但总体而言，由于微服务架构模式的分布式特性，该模式并不自然适合高性能应用程序。

- **可扩展性**

评级：高

分析：由于应用程序被分为单独部署的单元，因此每个服务组件都可以单独扩展，从而可以对应用程序进行微调扩展。例如，股票交易应用程序的管理区域可能不需要扩展，因为该功能的用户量较低，但交易安置服务组件可能需要扩展，因为大多数交易应用程序为此需要高吞吐量。功能。

- **易于开发**

评级：高

分析：由于功能被隔离为单独且不同的服务组件，因此由于范围更小且隔离，开发变得更加容易。开发人员对一个服务组件进行影响其他服务组件的更改的可能性要小得多，从而减少了开发人员或开发团队之间所需的协调。

天基架构

大多数基于 Web 的业务应用程序都遵循相同的一般请求流程：来自浏览器的请求到达 Web 服务器，然后到达应用程序服务器，最后到达数据库服务器。虽然这种模式对于一小部分用户来说效果很好，但随着用户负载的增加，瓶颈开始出现，首先是在 Web 服务器层，然后是应用程序服务器层，最后是数据库服务器层。对基于用户负载增加的瓶颈的通常响应是横向扩展 Web 服务器。这相对简单且便宜，有时可以解决瓶颈问题。然而，在大多数高用户负载的情况下，扩展 Web 服务器层只会将瓶颈转移到应用程序服务器。扩展应用程序服务器可能比 Web 服务器更加复杂和昂贵，并且通常只是将瓶颈移至数据库服务器，而数据库服务器的扩展更加困难和昂贵。即使您可以扩展数据库，最终得到的也是三角形拓扑，三角形最宽的部分是 Web 服务器（最容易扩展），最小的部分是数据库（最难扩展）。

在任何具有极大并发用户负载的大容量应用程序中，数据库通常是可以并发处理的事务数量的最终限制因素。尽管各种缓存技术和数据库扩展产品有助于解决这些问题，但事实仍然是，针对极端负载扩展正常应用程序是一个非常困难的提议。

基于空间的架构模式是专门为解决可扩展性和并发问题而设计的。对于具有可变且不可预测的并发用户量的应用程序来说，它也是一种有用的架构模式。从架构上解决极端和可变的可扩展性问题通常是比尝试横向扩展数据库或将缓存技术改造为不可扩展架构更好的方法。

模式描述

基于空间的模式（有时也称为云架构模式）最大限度地减少了限制应用程序扩展的因素。该模式的名称来源于元组空间的概念，即分布式共享内存的想法。通过消除中央数据库约束并使用复制的内存数据网格来实现高可扩展性。应用程序数据保存在内存中并在所有活动处理单元之间复制。处理单元可以随着用户负载的增加和减少而动态启动和关闭，从而解决可变的可扩展性。由于没有中央数据库，因此消除了数据库瓶颈，从而在应用程序内提供近乎无限的可扩展性。

大多数适合这种模式的应用程序都是标准网站，它们接收来自浏览器的请求并执行某种操作。竞价拍卖网站就是一个很好的例子。该网站通过浏览器请求不断接收互联网用户的出价。应用程序将接收特定项目的出价，用时间戳记录该出价，并更新该项目的最新出价信息，并将信息发送回浏览器。

该架构模式有两个主要组件：处理单元和虚拟化中间件。[图5-1](#) 说明了基本的天基架构模式及其主要架构组件。

处理单元组件包含应用程序组件（或应用程序组件的一部分）。这包括基于 Web 的组件以及后端业务逻辑。处理单元的内容根据应用程序的类型而变化 - 较小的基于 Web 的应用程序可能会部署到单个处理单元中，而较大的应用程序可能会根据应用程序的功能区域将应用程序功能拆分为多个处理单元。处理单元通常包含应用程序模块，以及内存中的数据网格和用于故障转移的可选异步持久存储。它还包含一个复制引擎，虚拟化中间件使用该引擎将一个处理单元所做的数据更改复制到其他活动处理单元。

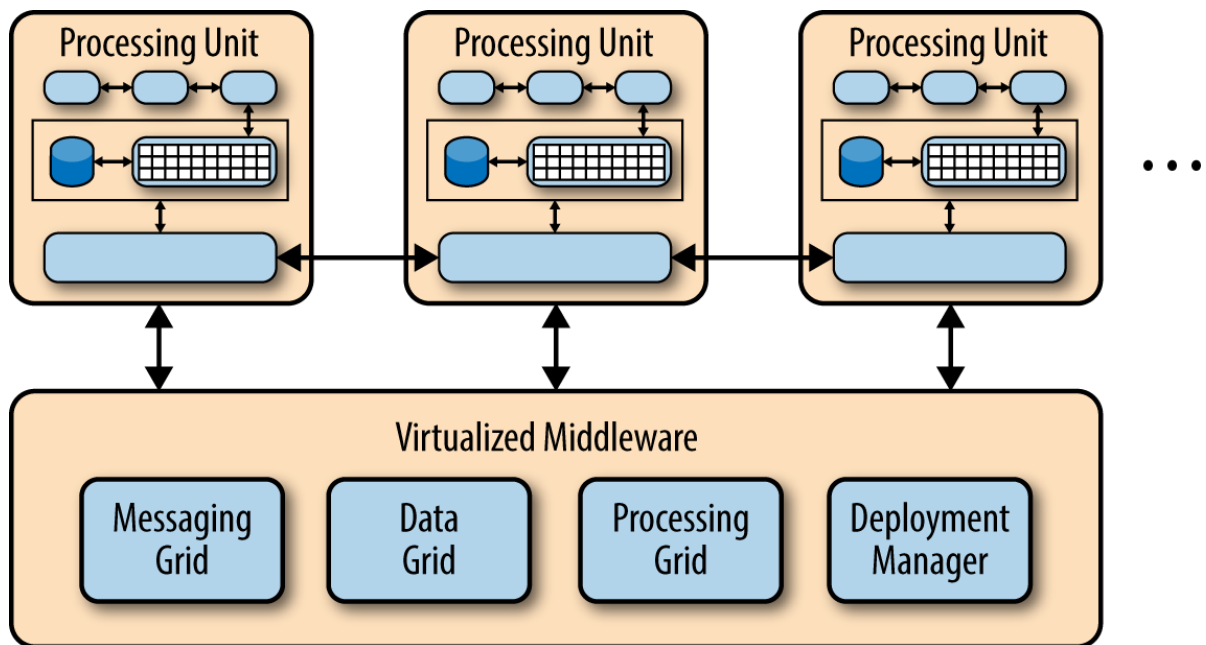


图 5-1.天基架构模式

虚拟化中间件组件处理内务管理和通信。它包含控制数据同步和请求处理各个方面的组件。虚拟化中间件包括消息传递网格、数据网格、处理网格和部署管理器。这些组件将在下一节中详细描述，可以自定义编写或作为第三方产品购买。

模式动力学

基于空间的架构模式的魔力在于虚拟化中间件组件和每个处理单元中包含的内存数据网格。[图 5-2](#) 显示了典型的处理单元架构，其中包含应用程序模块、内存数据网格、用于故障转移的可选异步持久存储以及数据复制引擎。

虚拟化中间件本质上是架构的控制器，管理请求、会话、数据复制、分布式请求处理和流程单元部署。虚拟化中间件中有四个主要的架构组件：消息传递网格、数据网格、处理网格和部署管理器。

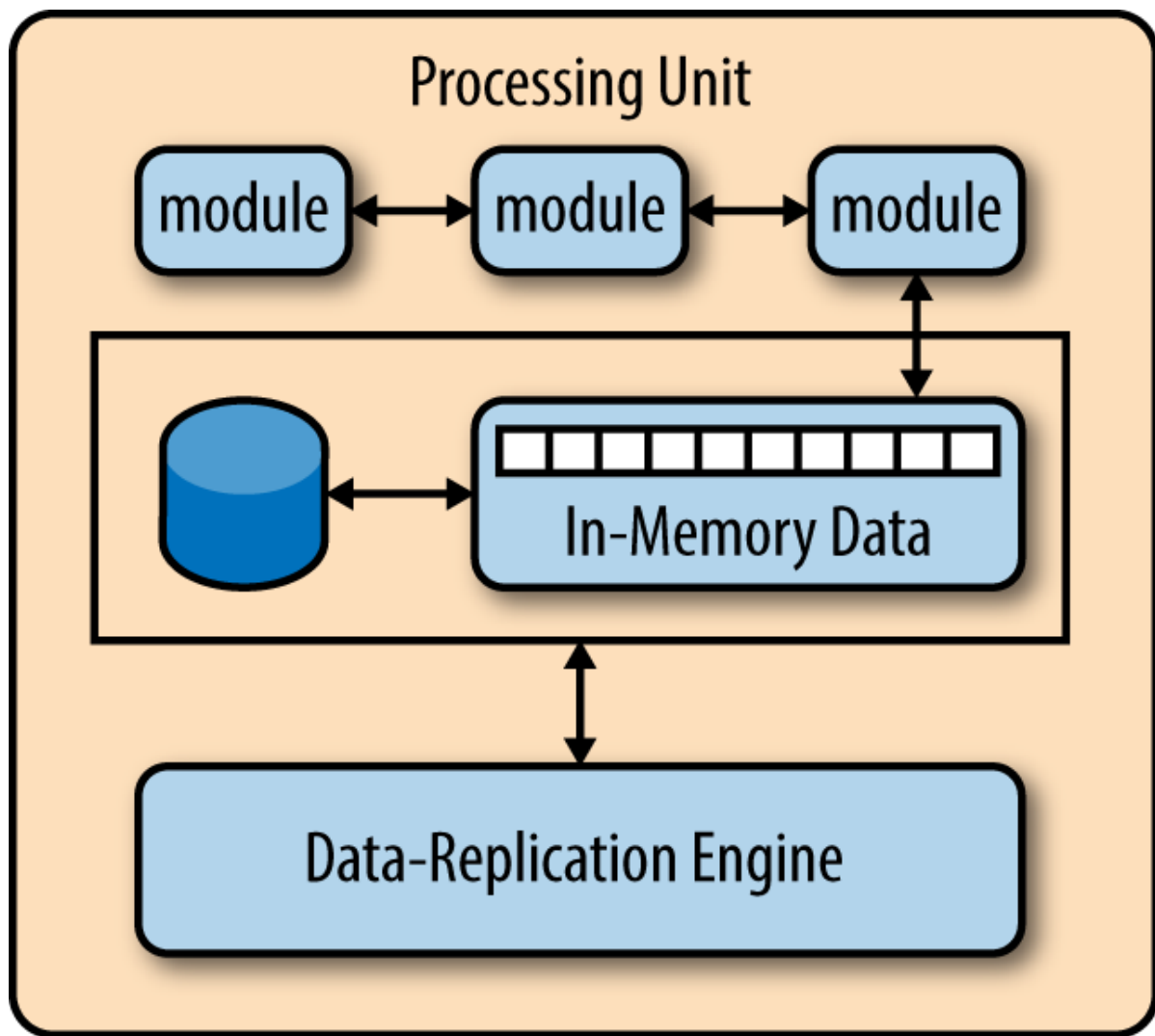


图 5-2. 处理单元组件

消息传递网格

消息传递网格如图 5-3 所示，管理输入请求和会话信息。当请求进入虚拟化中间件组件时，消息传递网格组件确定哪些活动处理组件可用于接收该请求并将该请求转发到这些处理单元之一。消息传递网格的复杂性范围可以从简单的循环算法到更复杂的下一个可用算法，该算法跟踪哪个处理单元正在处理哪个请求。

数据网格

数据网格组件可能是该模式中最重要和最关键的组件。数据网格与每个处理单元中的数据复制引擎交互，以在数据更新发生时管理处理单元之间的数据复制。由于消息传递网格可以将请求转发到任何可用的处理单元，因此每个处理单元在其内存数据网格中包含完全相同的数据至关重要。虽然图 5-4 显示了处理单元之间的同步数据复制，但实际上这是并行异步完成的，而且速度非常快，有时只需几微秒（百万分之一秒）即可完成数据同步。

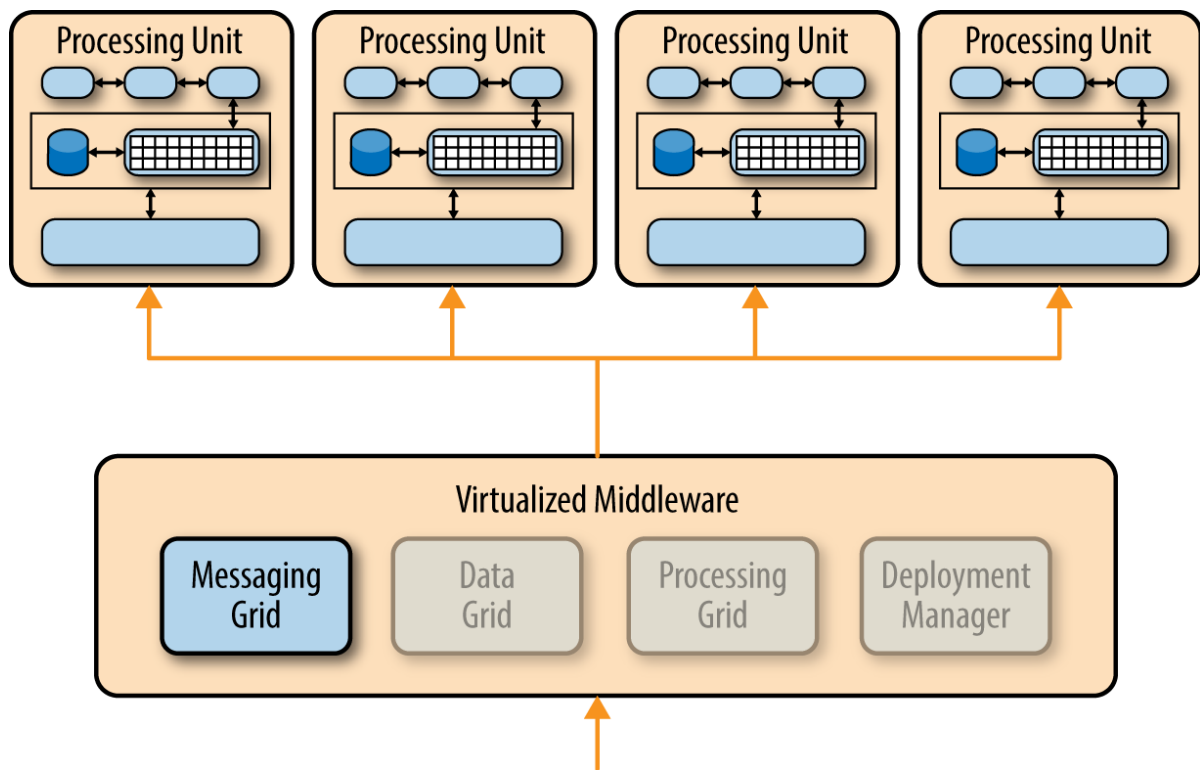


图 5-3. 消息传递网络组件

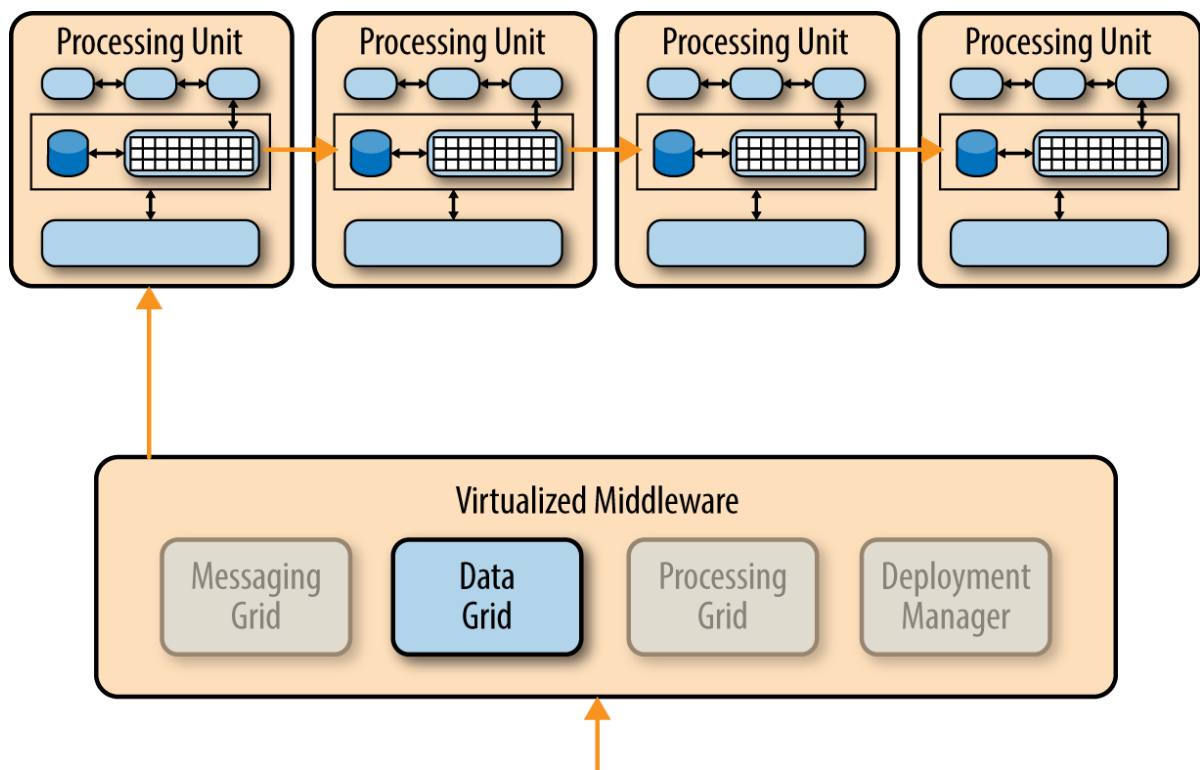


图 5-4. 数据网格组件

处理网络

如图 5-5 所示，处理网格是虚拟化中间件中的一个可选组件，当存在多个处理单元（每个处理单元处理应用程序的一部分）时，它管理分布式请求处理。如果传入的请求需要处理单元类型（例如，订单处理单元和客户处理单元）之间的协调，则处理网格将在这两个处理单元之间协调和协调该请求。

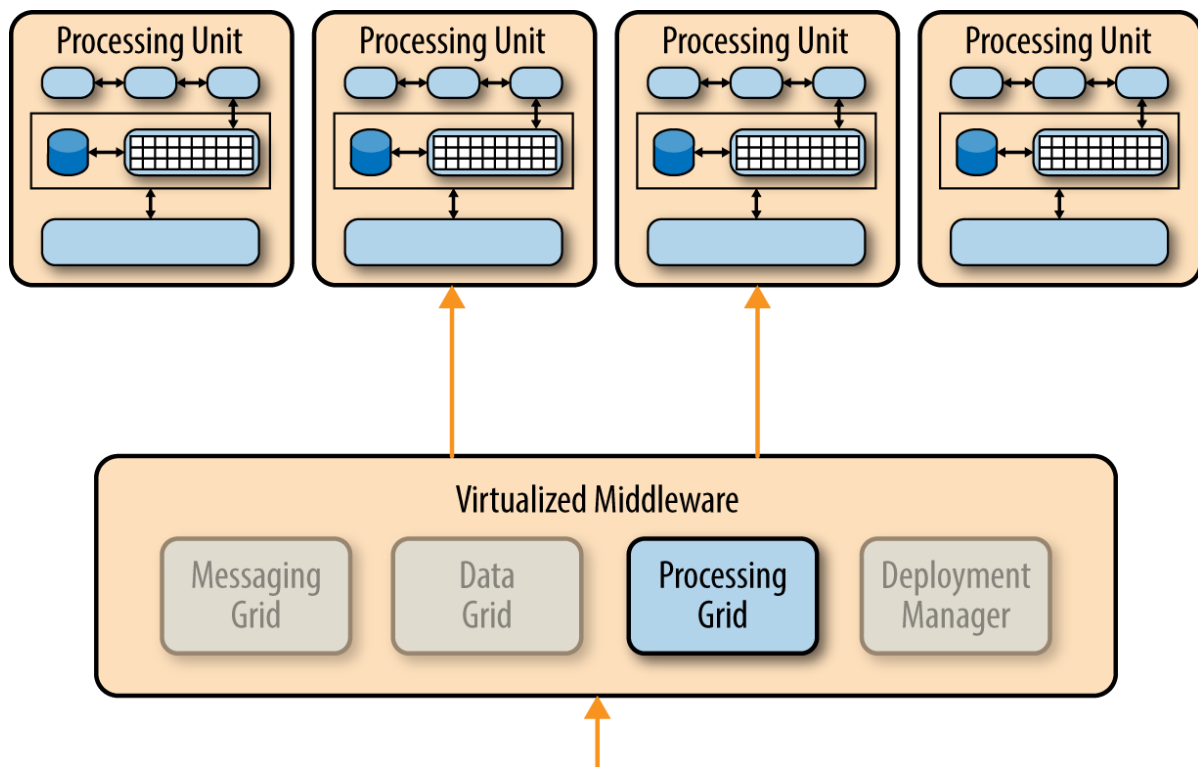


图 5-5.处理网格组件

部署经理

部署管理器组件根据负载条件管理处理单元的动态启动和关闭。该组件持续监视响应时间和用户负载，并在负载增加时启动新的处理单元，并在负载减少时关闭处理单元。它是在应用程序中实现可变可扩展性需求的关键组件。

注意事项

天基架构模式是一种实施起来复杂且昂贵的模式。对于具有可变负载的小型基于 Web 的应用程序（例如社交媒体网站、投标和拍卖网站）来说，这是一个很好的架构选择。然而，它不太适合具有大量操作数据的传统大型关系数据库应用。

尽管基于空间的架构模式不需要集中式数据存储，但通常会包含一个集中式数据存储来执行初始内存中数据网格加载并异步保存处理单元所做的数据更新。创建单独的分区以将易失性和广泛使用的事务数据与非活动数据隔离开也是一种常见的做法，以减少每个处理单元内内存数据网格的内存占用。

值得注意的是，虽然此模式的替代名称是基于云的架构，但处理单元（以及虚拟化中间件）不必驻留在基于云的托管服务或 PaaS（平台即服务）上。它可以轻松地驻留在本地服务器上，这就是我更喜欢“基于空间的架构”这个名称的原因之一。

从产品实现的角度来看，您可以通过第三方产品（例如 GemFire、JavaSpaces、GigaSpaces、IBM Object Grid、nCache 和 Oracle Coherence）来实现此模式中的许多架构组件。由于此模式的实现在成本和功能（特别是数据复制时间）方面差异很大，因此作为架构师，您应该在做出任何产品选择之前首先确定您的具体目标 and 需求。

模式分析

下表包含对基于空间的架构模式的常见架构特征的评级和分析。每个特征的评级基于该特征作为基于模式的典型实现的能力的自然趋势，以及该模式通常为人所知的内容。有关此模式与本报告中其他模式的关系的并排比较，请参阅本报告末尾的[模式分析摘要](#)。

- 整体敏捷性

评级：高

分析：整体敏捷性是指对不断变化的环境做出快速响应的能力。由于处理单元（应用程序的部署实例）可以快速启动和关闭，因此应用程序可以很好地响应与用户负载（环境变化）增加或减少相关的变化。由于应用程序规模较小且模式的动态特性，使用此模式创建的架构通常能够很好地响应编码更改。

- **易于部署**

评级：高

分析：虽然天基架构通常不是解耦和分布式的，但它们是动态的，并且复杂的基于云的工具允许将应用程序轻松地“推送”到服务器，从而简化部署。

- **可测试性**

评级：低

分析：在测试环境中实现非常高的用户负载既昂贵又耗时，因此很难测试应用程序的可扩展性。

- **表现**

评级：高

分析：高性能是通过此模式中内置的内存中数据访问和缓存机制来实现的。

- **可扩展性**

评级：高

分析：高可扩展性来自于对集中式数据库的依赖很少或不依赖这一事实，因此从本质上消除了可扩展性等式中的这一限制瓶颈。

- **易于开发**

评级：低

分析：复杂的缓存和内存数据网格产品使得这种模式的开发相对复杂，主要是因为对用于创建此类架构的工具和产品缺乏熟悉。此外，在开发这些类型的架构时必须特别小心，以确保源代码中的任何内容不会影响性能和可扩展性。

模式分析总结

[图 1-1](#) 总结了本报告中描述的每种架构模式的模式分析评分。此摘要将帮助您确定哪种模式最适合您的情况。例如，如果您主要关心的架构问题是可扩展性，您可以查看此图表，发现事件驱动模式、微服务模式和基于空间的模式可能是不错的架构模式选择。同样，如果您为应用程序选择分层架构模式，则可以参考图表来了解部署、性能和可扩展性可能是架构中的风险领域。

	Layered	Event-driven	Microkernel	Microservices	Space-based
Overall Agility	↓	↑	↑	↑	↑
Deployment	↓	↑	↑	↑	↑
Testability	↑	↓	↑	↑	↓
Performance	↓	↑	↑	↓	↑
Scalability	↓	↑	↓	↑	↑
Development	↑	↓	↓	↑	↓

图 1-1.模式分析总结

虽然此图表将帮助指导您选择正确的模式，但在选择架构模式时还需要考虑更多因素。您必须分析环境的各个方面，包括基础设施支持、开发人员技能集、项目预算、项目截止日期和应用程序大小（仅举几例）。选择正确的架构模式至关重要，因为架构一旦到位，就很难（而且成本高昂）进行更改。